Abstract Factory
Strategy
State
Chain of Responsibility

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 10

Abstract Factory Pattern

WHEN YOU NEED TO CREATE FAMILIES OF OBJECTS INSTEAD OF A SINGLE OBJECT.

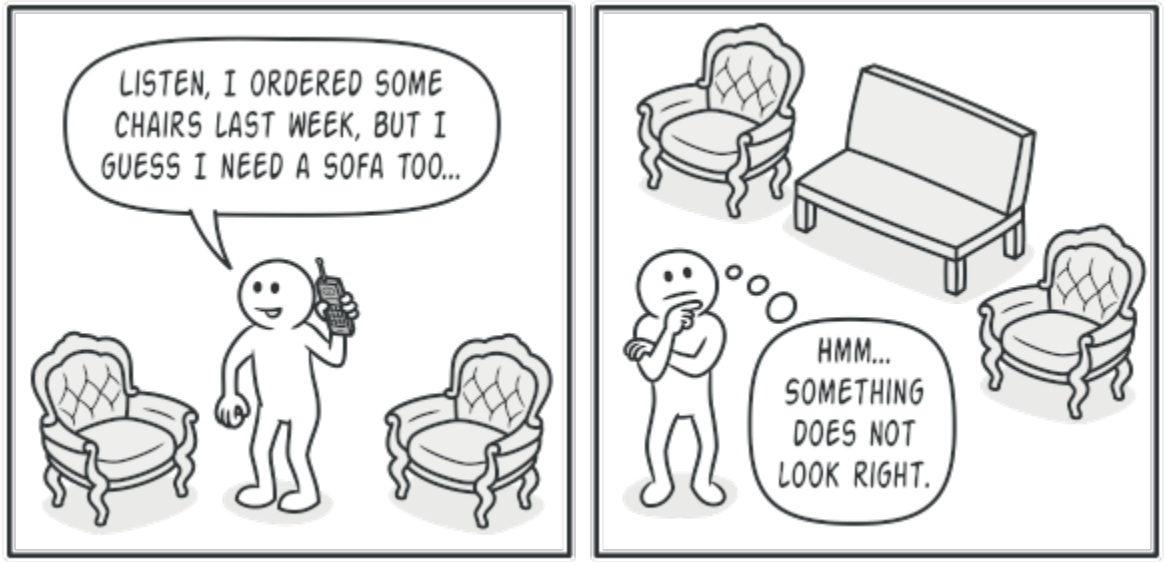
Create Families of Products.

Abstract Factory is a creational design pattern that lets you produce **families of related objects without specifying their concrete classes**.

In a Factory pattern, each factory creates a **single product**.

In an Abstract Factory pattern, each factory creates a **group of related products**

This is useful if our code **requires objects in specific groups** that are **compatible** with each other.



Create Families of Products.

The Abstract Factory builds upon the Factory Pattern.

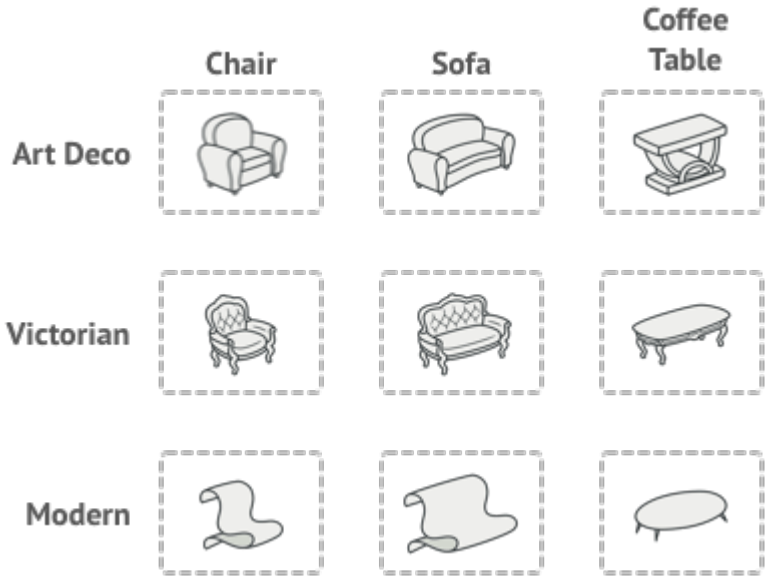
No, making the base factory abstract is not enough to make an Abstract Factory Pattern.

So here's a new scenario.

Say we have a bunch of objects **that are related to each other** somehow, they make sense together. Let's call this a **"product family"**.

Now say there are **different variations** of these product families.

The Abstract Factory Pattern modifies the Factory Pattern to accommodate this type of complexity.



Product Family (related products):

Chair + Sofa + CoffeeTable

Product Family Variations:

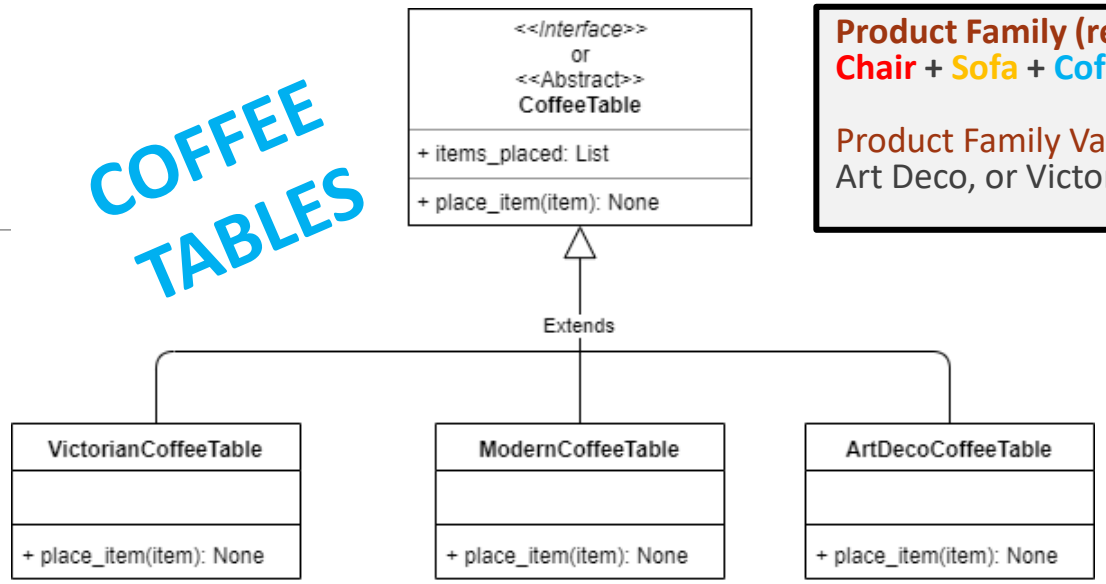
Art Deco, or Victorian, or Modern

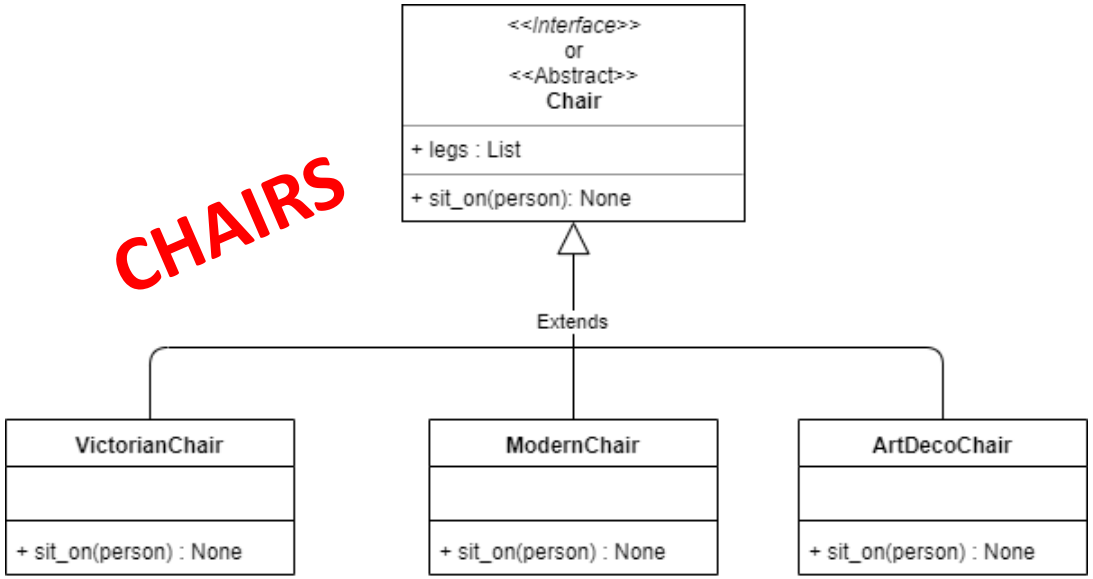
Abstract Factory

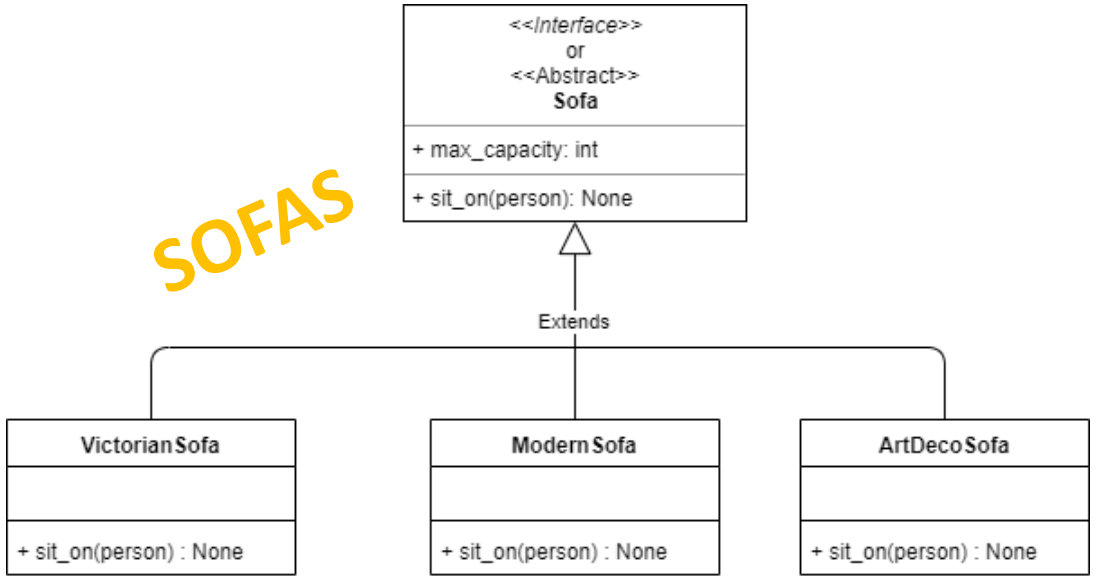
In the abstract factory pattern, instead of one product hierarchy, you would have a **different hierarchy for each distinct product in the product family**.

Product Family (related products):
Chair + Sofa + CoffeeTable

Product Family Variations:
Art Deco, or Victorian, or Modern







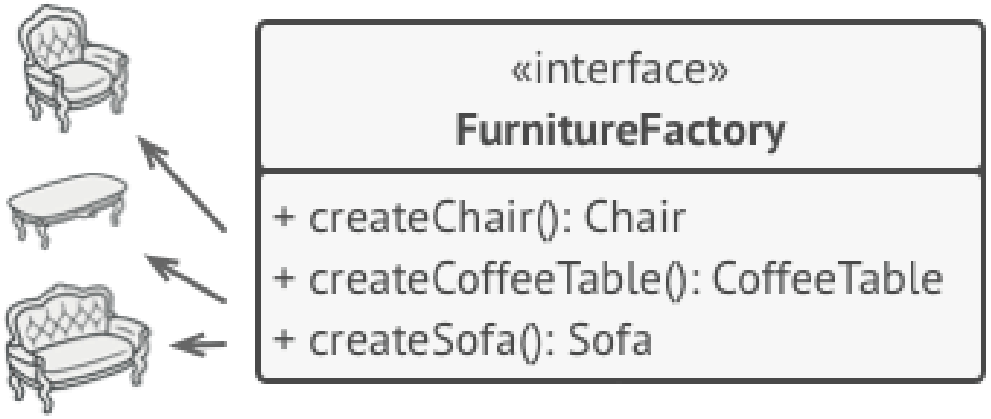
Abstract Factory

Product Family (related products):
Chair + Sofa + CoffeeTable

Product Family Variations:
Art Deco, or Victorian, or Modern

Instead of having a factory that creates one item, we would have **a factory that creates a product family**.

A factory creates an instance of each object in the family and those objects are guaranteed to be compatible.



Abstract Factory

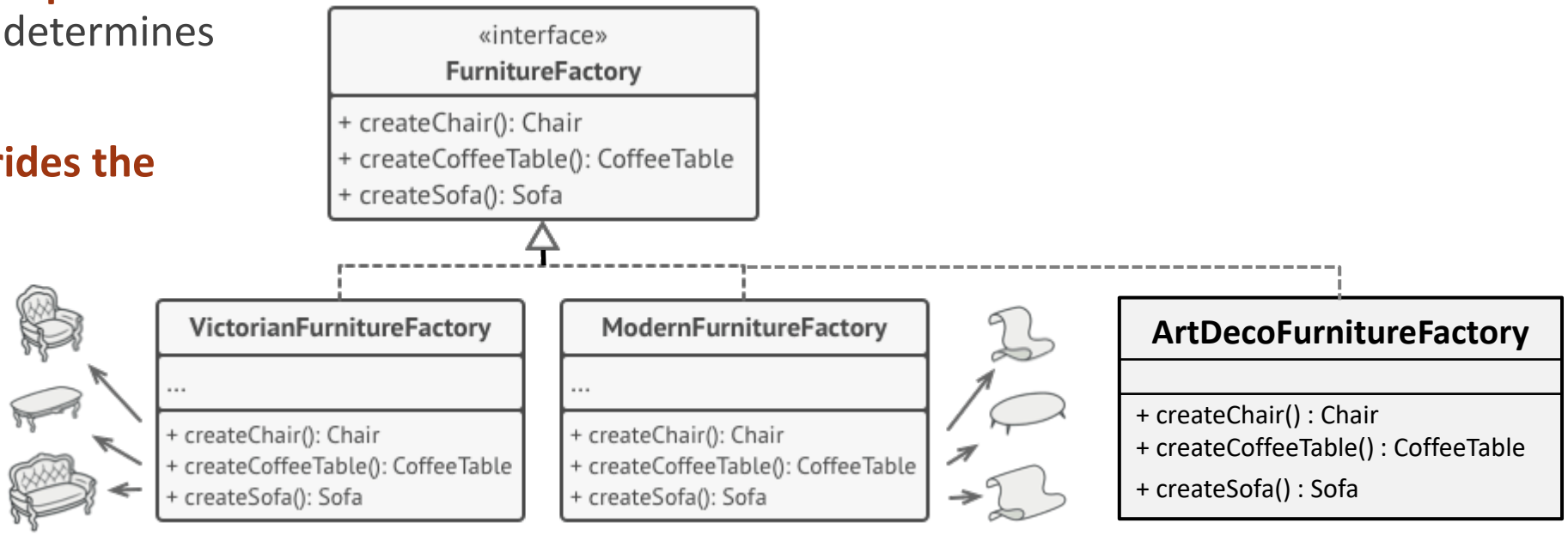
Product Family (related products):
Chair + Sofa + CoffeeTable

Product Family Variations:
Art Deco, or Victorian, or Modern

Just like the factory pattern, we have to extend our Base Factory class (Furniture Factory) and create a **Concrete Factory.**

Each Concrete Factory now represents a theme/variety. This is what determines the **compatibility**.

Each Concrete Factory **overrides the creation methods** to return objects from the **same Product Family.**



Abstract Factory

Product Family (related products):
Chair + Sofa + CoffeeTable

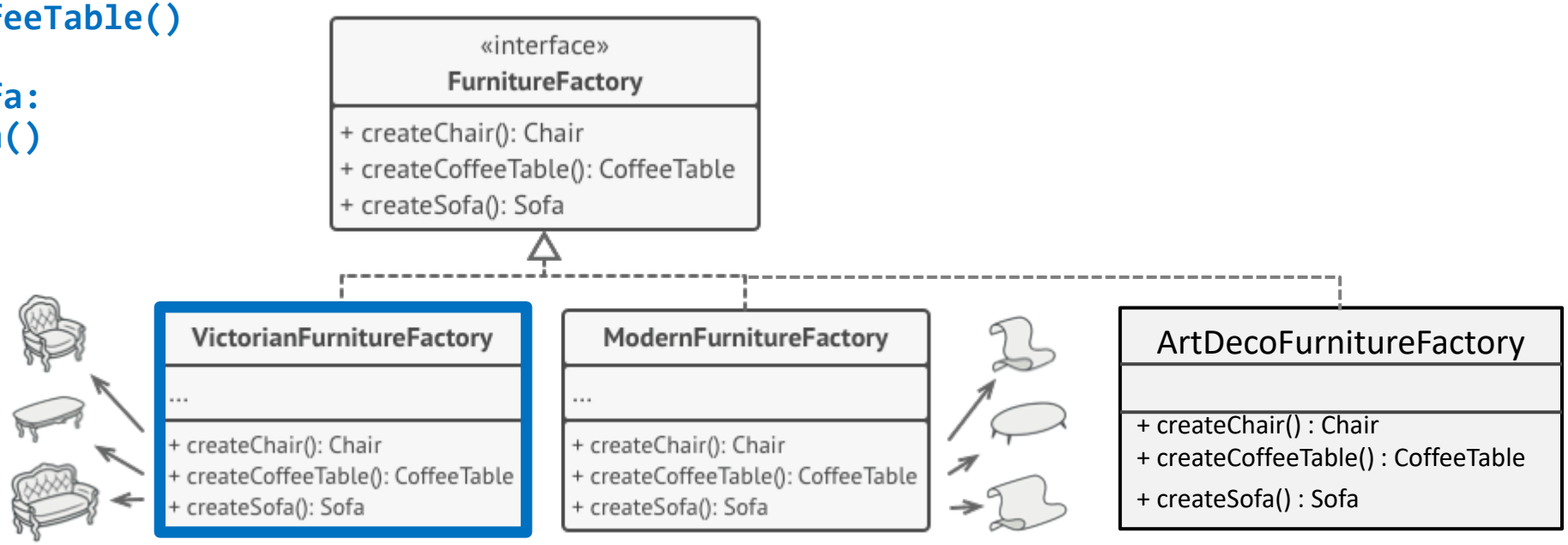
Product Family Variations:
Art Deco, or Victorian, or Modern

```
class VictorianFurnitureFactory(FurnitureFactory):

    def create_chair() -> Chair:
        return VictorianChair()

    def create_coffee_table() -> CoffeeTable:
        return VictorianCoffeeTable()

    def create_sofa() -> Sofa:
        return VictorianSofa()
```



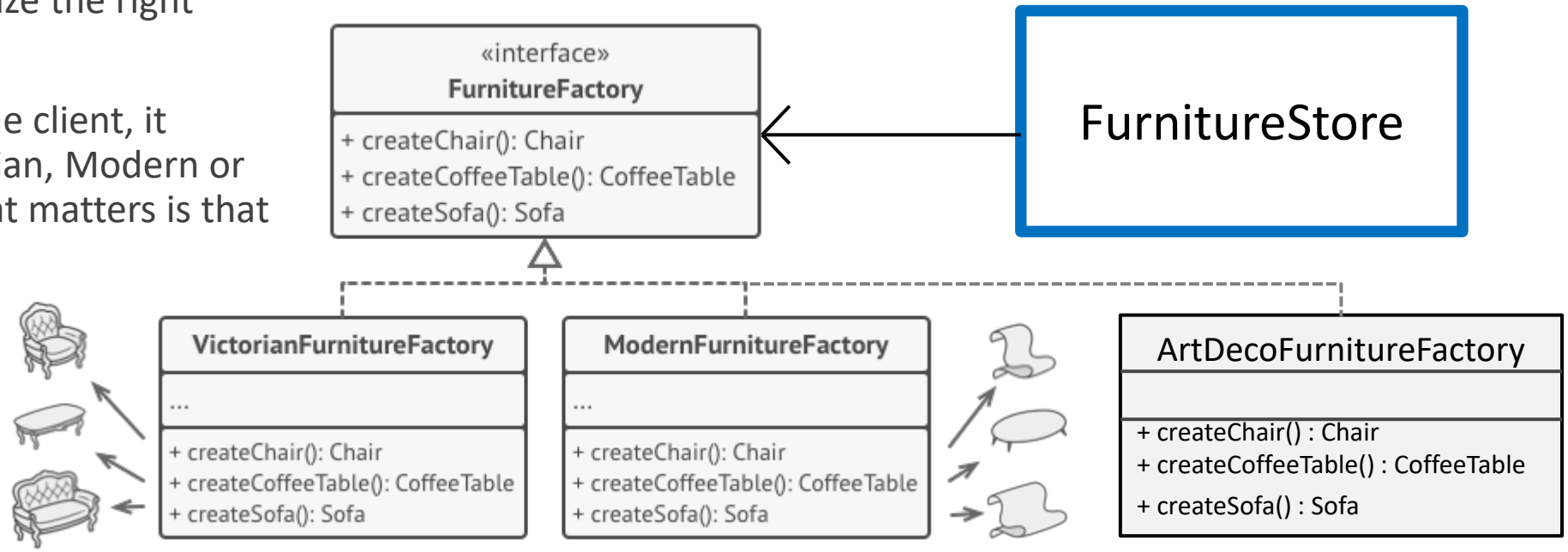
Abstract Factory

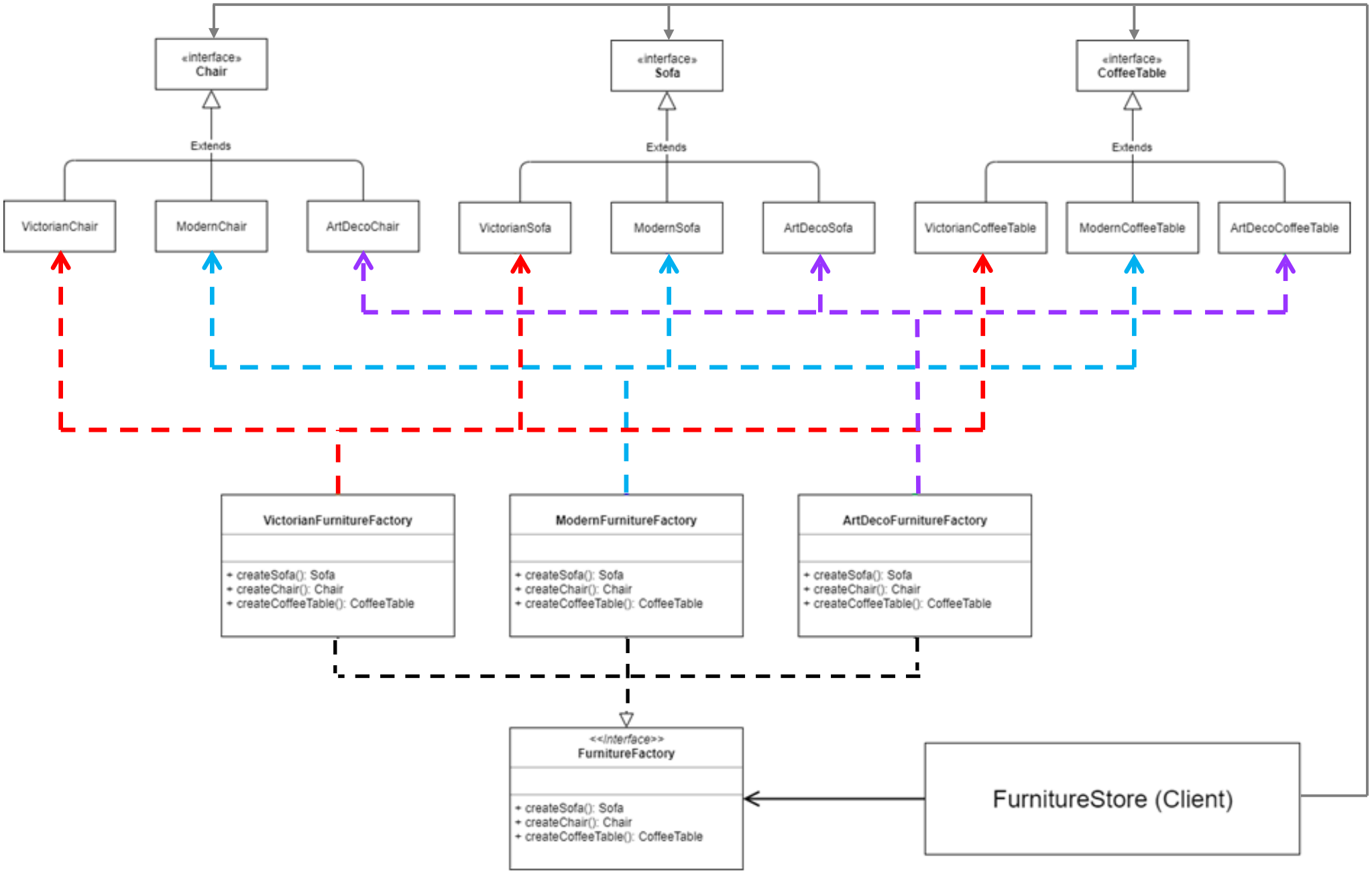
Now we just need to add in the **client**.

The client expects an object with FurnitureFactory Interface.

We can have a **controller** initialize the right factory and pass it to the client.

Once the factory is passed to the client, it shouldn't matter if it is a Victorian, Modern or ArtDeco Furniture Factory. What matters is that it has the right interface.



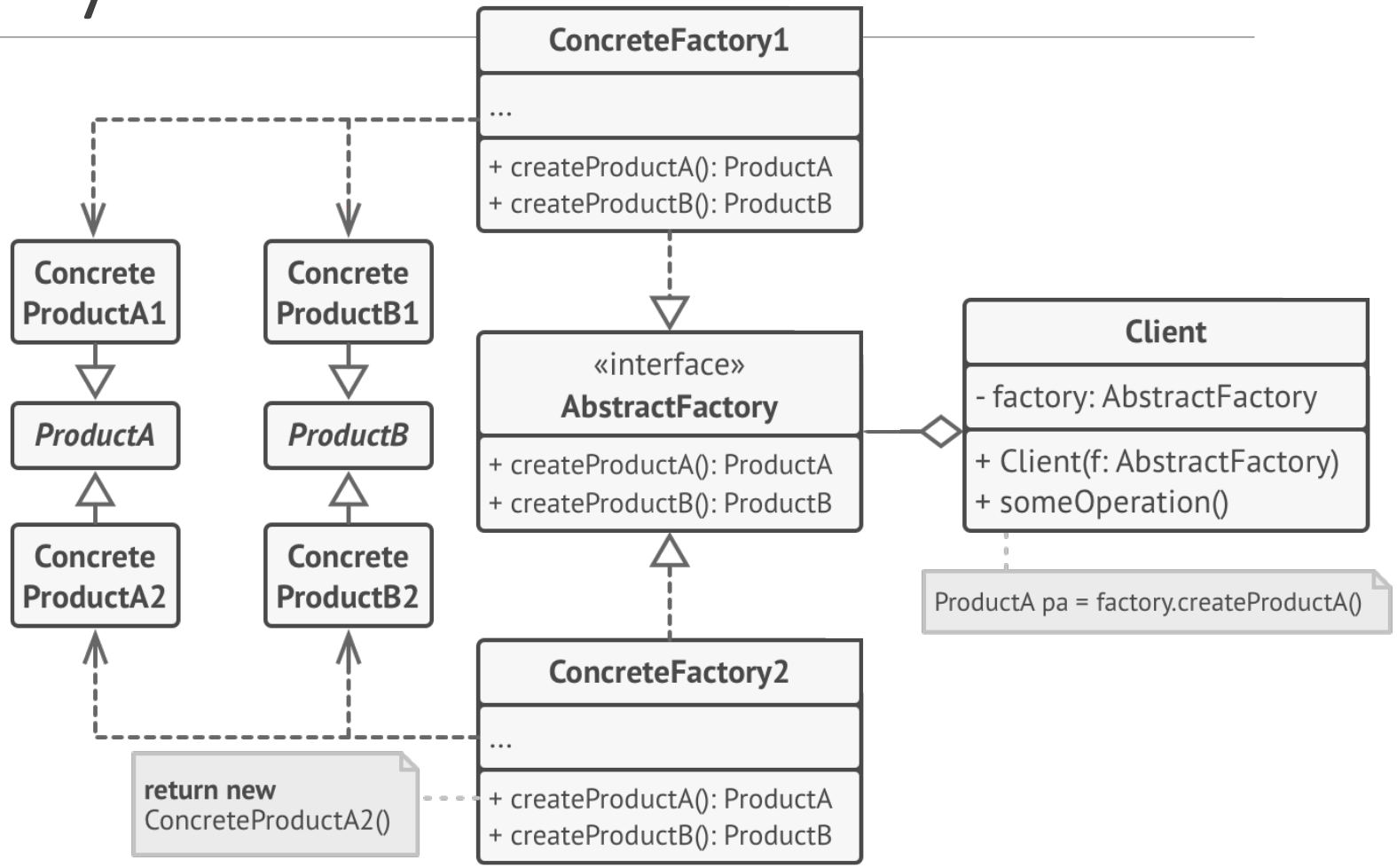


Abstract Factory – Putting it all together

Abstract Factory

It's essentially very similar to the Factory Pattern.

It just looks complicated since we are dealing with more classes due to the large number of products.



Abstract Factory – Let's Make one!

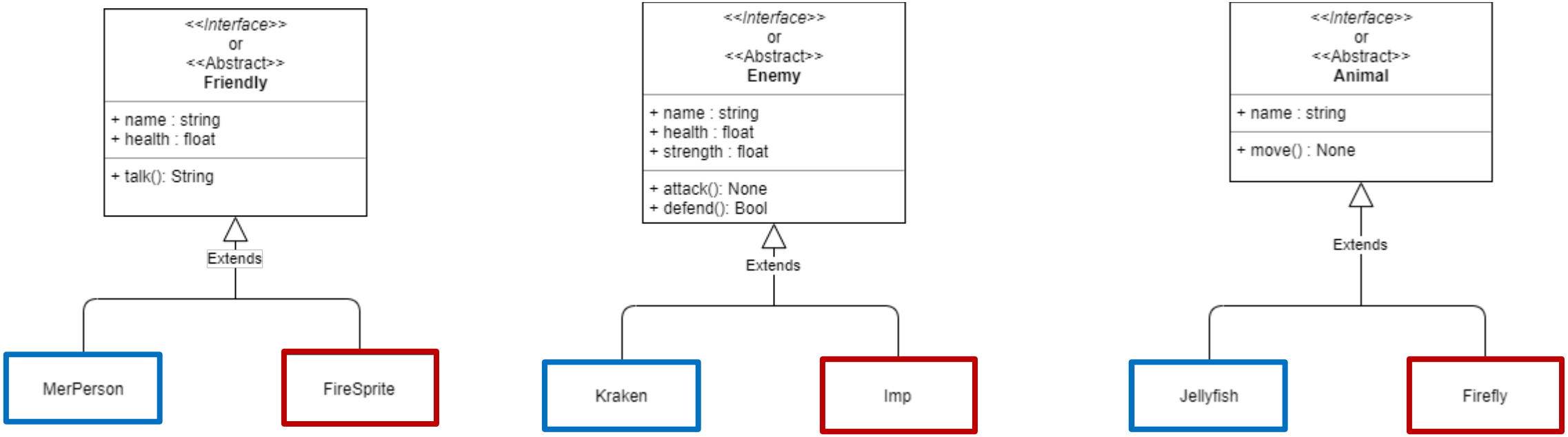
Say we were working on a game that wanted to spawn groups of characters (Friendlies, Enemies and Animals) in different worlds.

Each world would have characters following various themes.

The first step is to create a matrix identifying our **Product Families** (groups of related objects) in one dimension, and **varieties** (themes) in another dimension.

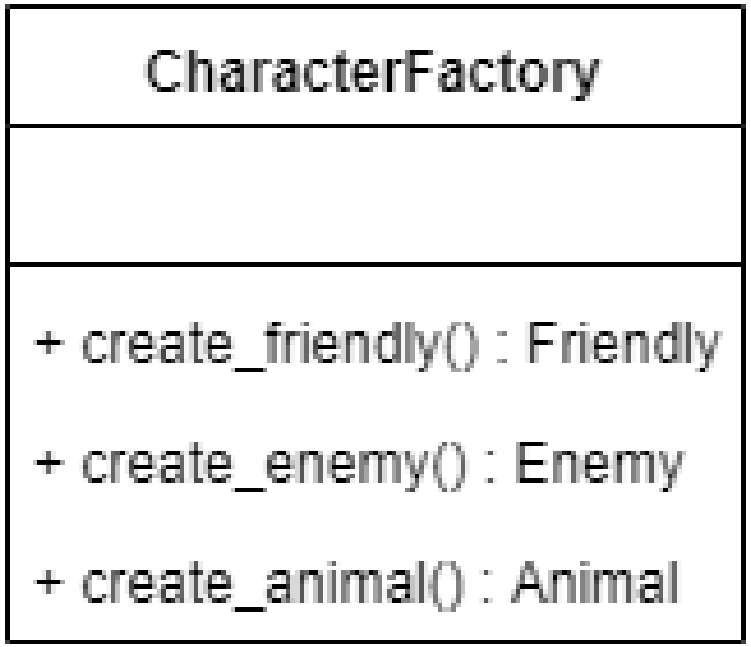
Theme/Product	Friendlies	Enemies	Animals
Aquatica	MerPerson	Kraken	Jellyfish
Firelands	FireSprite	Imp	Firefly

Step 1: Define the inheritance hierarchy for each product in the product family.

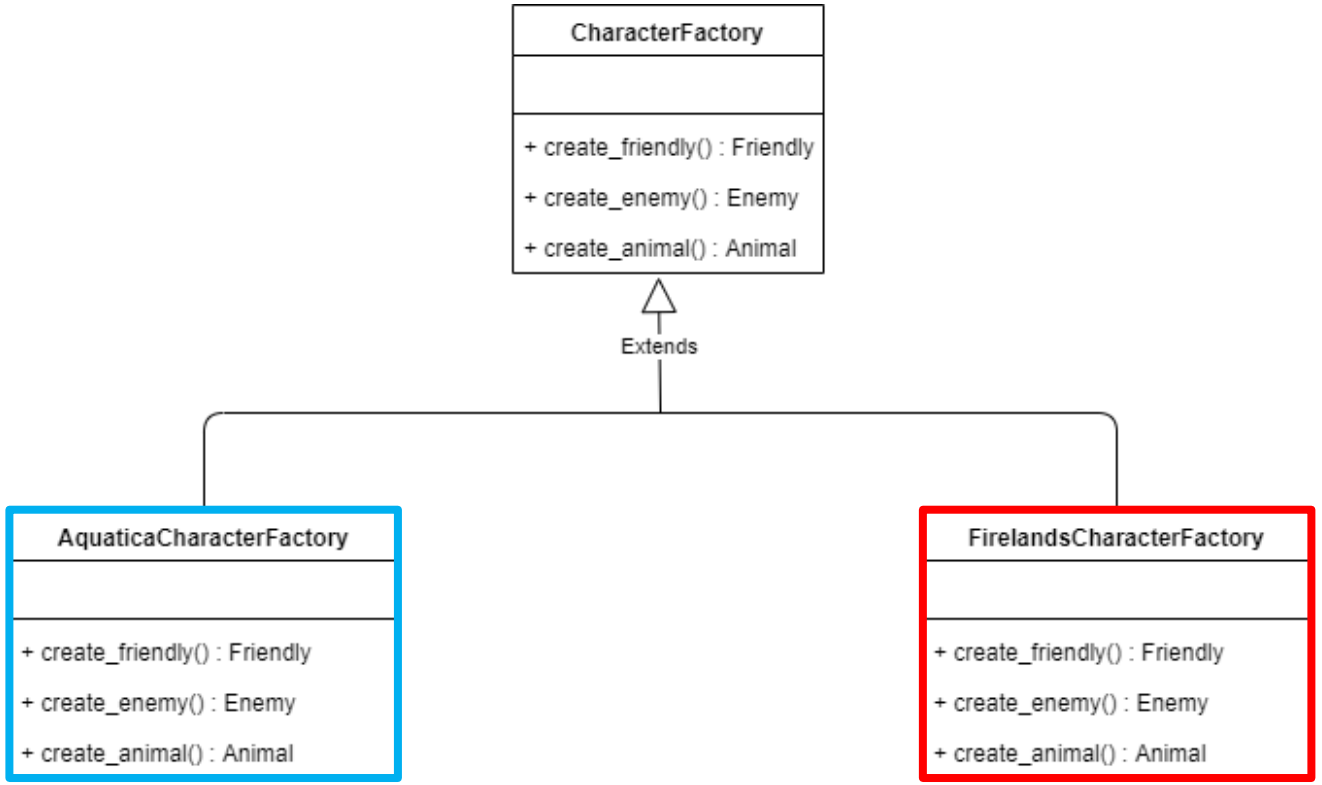


Theme/Product	Friendlies	Enemies	Animals
Aquatica	MerPerson	Kraken	Jellyfish
Firelands	FireSprite	Imp	Firefly

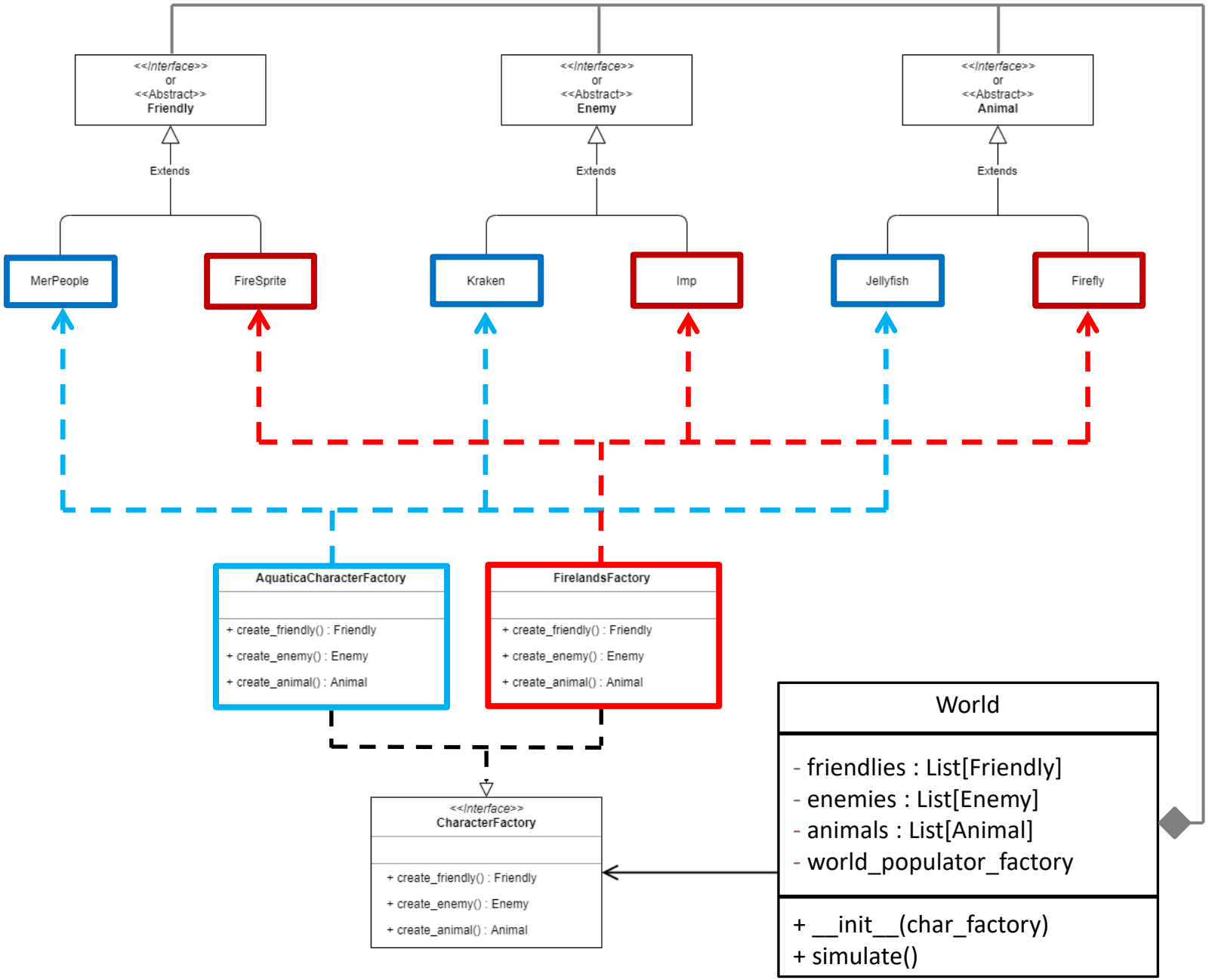
Step 2: Define a Factory base class that can create a Product Family



Step 3: Extend the Factory base class with Concrete Factories that can create Varieties.



Step 4: Put it all together



game_abstract_factory.py

Abstract Factory Pattern: Why and When do we use it

- Use this pattern if the code needs to work with **families of related objects**.
- If the **exact number of Product Families are unknown** and you will have to **add in new ones in the future**.
- Single Responsibility, Open/Closed Principle, Liskov Substitution and Dependency Inversion Principle.

Abstract Factory Pattern – Disadvantages

- There are a **lot of classes** and interfaces in play. This can make the **code complex and hard to debug**.
- Sometimes the **classes can get artificial**. You may decide to create a whole **new subclass for a very minor change** in the object creation process.

Strategy

WHEN YOU WANT TO BE ABLE TO CHANGE YOUR PLANS

Liskov Substitution Principle

In programming, the Liskov substitution principle states that if ***S*** is a subtype of ***T***, then objects of type ***T*** may be replaced (or substituted) with objects of type ***S***.

Or

Objects in a program should be replaceable with instances of their base types without altering the correctness of that program.

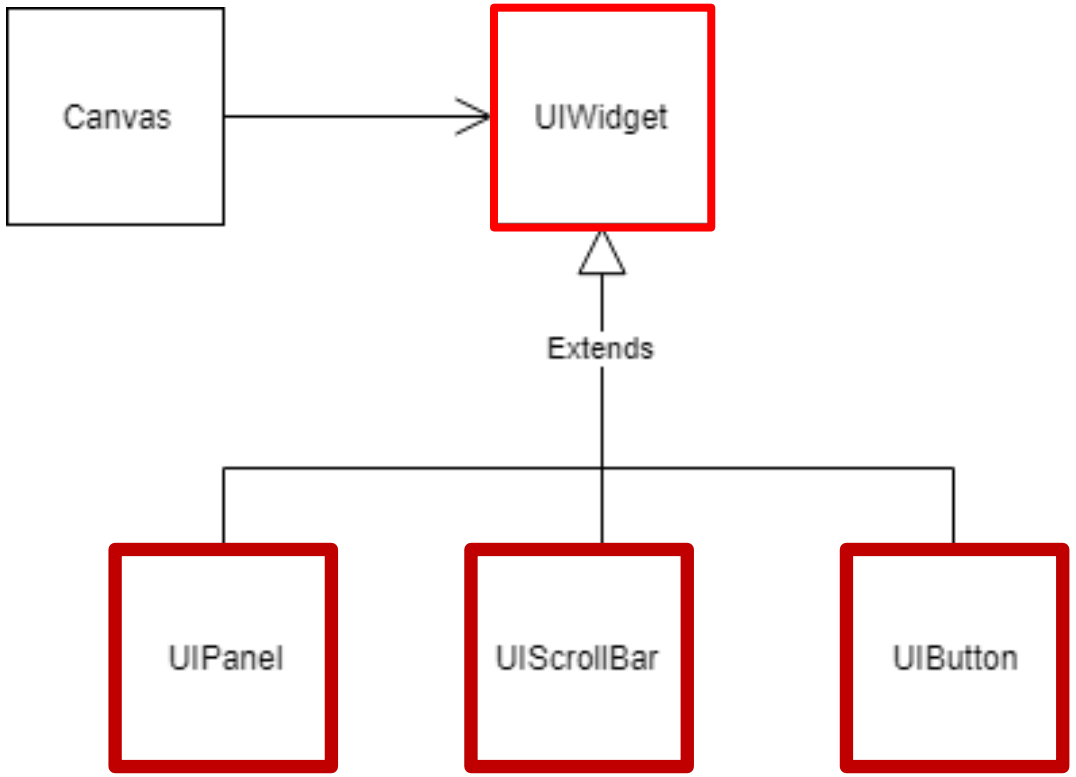
Simply put, you should be able to refer to all the different subclasses as its parent class type

In Python this is easy, since we don't worry about types!

```
class Canvas:

    def __init__(self, widget_list):
        self.ui_widgets = widget_list

    def draw_screen(self):
        for widget in self.ui_widgets:
            widget.draw()
```

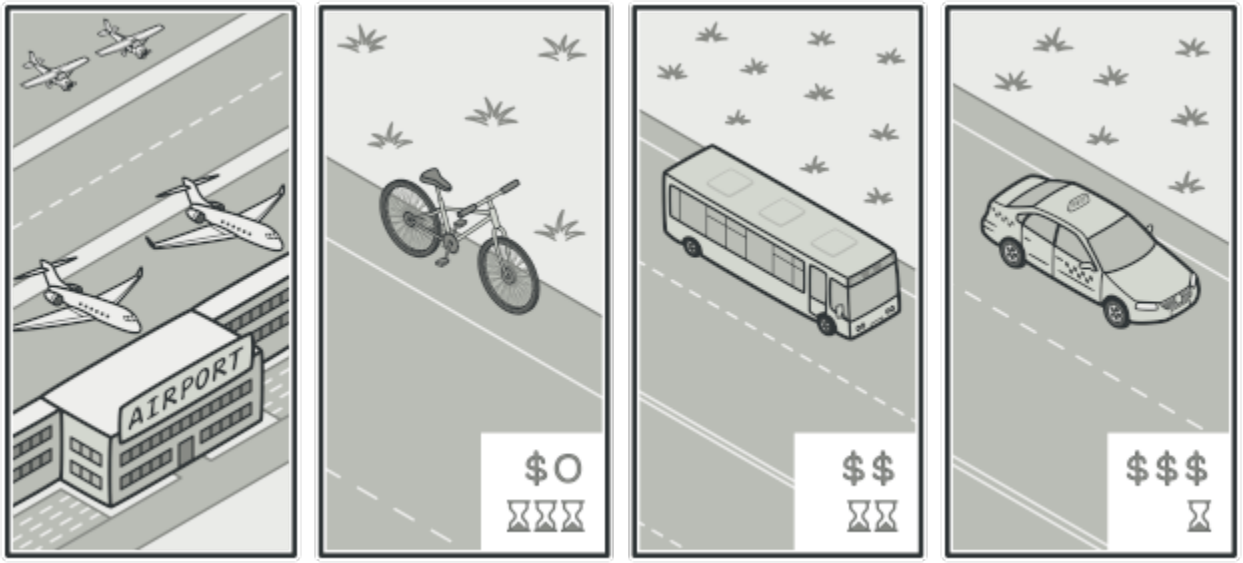


Wherever **widget** is used, we should be able to substitute its subclasses **UIPanel**, **UIScrollBar**, and **UIButton**

If we can not then the **Liskov substitution principle fails**. This means either the classes or the code calling the classes is implemented incorrectly

Strategy

- This is a really simple pattern that embodies Liskov's Substitution Principle
- In this pattern we create a **hierarchy of behaviours** that inherit from a base class
- We then use **composition** to be able to **swap behaviors** at run time.
- You have done this already at some stage or form
- For example, you want to get to the airport and there are many **strategies** to do so
  - ```
    def go_to_airport():
        self.transport_strategy.go()
    ```
- You can use a **car**, a **bike** or a **bus**. They all have similar behaviours with different implementations


    ```

Strategy Scenario

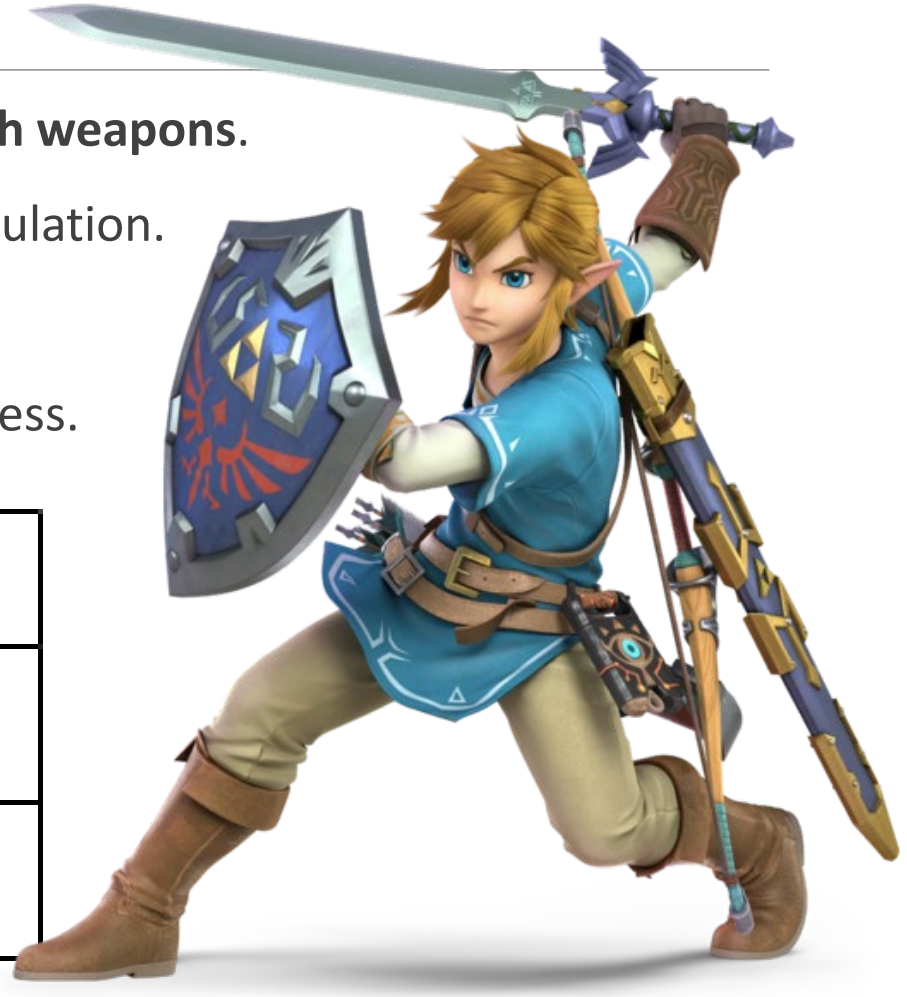
Say you have built a game that lets you **equip a character with weapons**.

This weapon has a specific **attack** animation and damage calculation.

This is handled by the attack method of your character class

You make the best game in the world and it's an instant success.

Character
+ attack(Enemy) : float



Strategy Scenario

Eager to continue pleasing your fans you decide to release an update with 2 new weapons!

You modified the Character class and added code to the attack method to contain the attack animations and damage calculations for these new weapons.

Over time you **added 5 more weapons.**

```
def attack(self, enemy):
    if self.weapon == Weapons.SWORD:
        # complicated sword attack logic
    if self.weapon == Weapons.SPEAR:
        # complicated spear attack logic
    if self.weapon == Weapons.BOW_AND_ARROW:
        # complicated bow and arrow attack logic
    if self.weapon == Weapons.CANNON:
        # complicated cannon attack logic
    if self.weapon == Weapons.MAGIC_WAND:
        # complicated magic wand attack logic
        ...
```

Strategy Scenario

Your Character class is huge. Any change to this causes errors.

Your team of developers keep having merge conflicts with this class. One developer is working on the character movement and you are working on character attack.

Single Responsibility Principle has been violated and so has open closed principle.

```
def attack(self, enemy):
    if self.weapon == Weapons.SWORD:
        # complicated sword attack logic
    if self.weapon == Weapons.SPEAR:
        # complicated spear attack logic
    if self.weapon == Weapons.BOW_AND_ARROW:
        # complicated bow and arrow attack logic
    if self.weapon == Weapons.CANNON:
        # complicated cannon attack logic
    if self.weapon == Weapons.MAGIC_WAND:
        # complicated magic wand attack logic
    ...
```

Strategy Scenario – Liskov Substitution Principle to the Rescue!

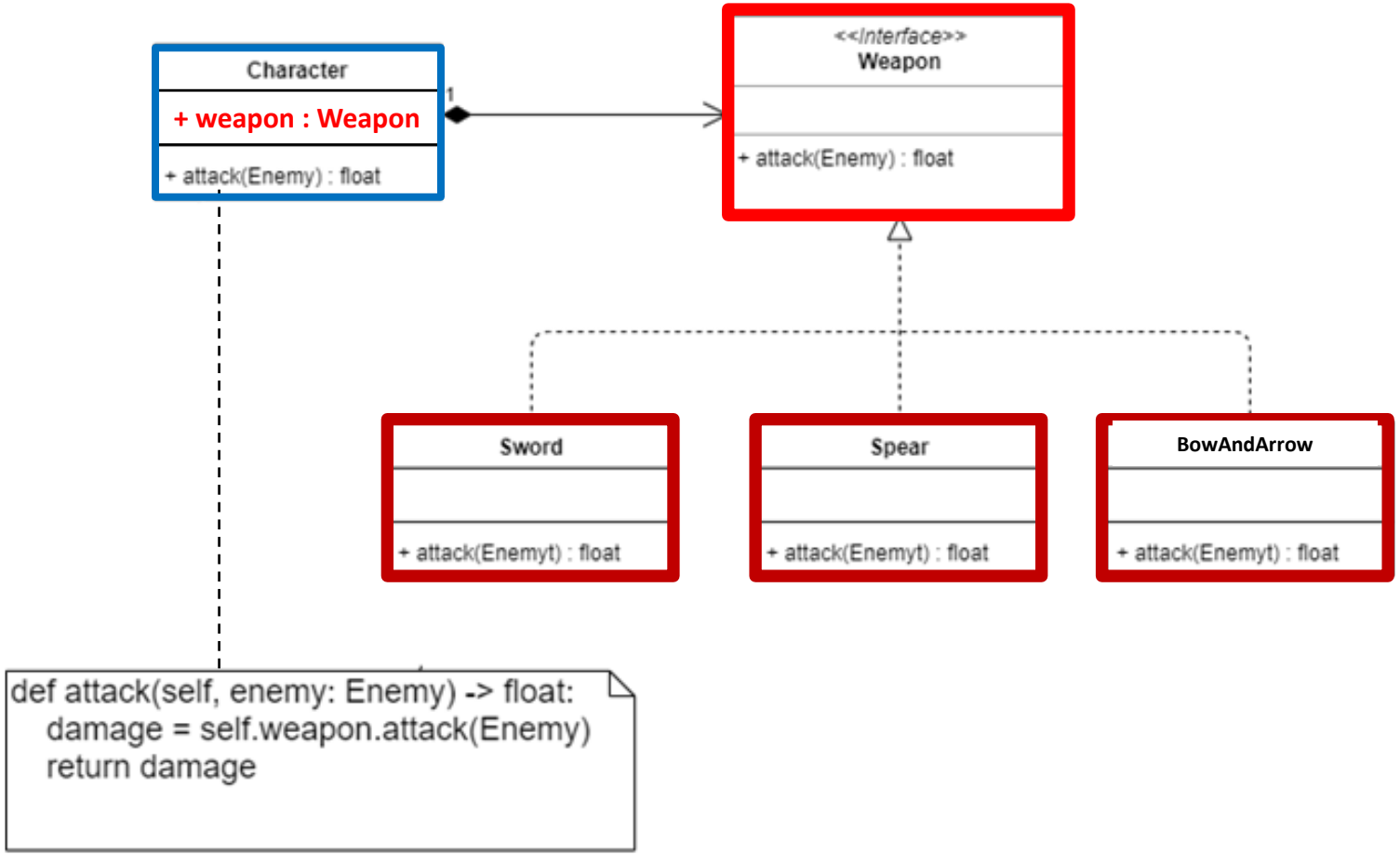
What if we used **composition** instead.

We can:

1) Separate the attack behaviour into its own class (Weapon)
Solves Single Responsibility Principle

2) Inherit from Weapon to implement **different kinds of weapons.**
Solves Open Closed Principle

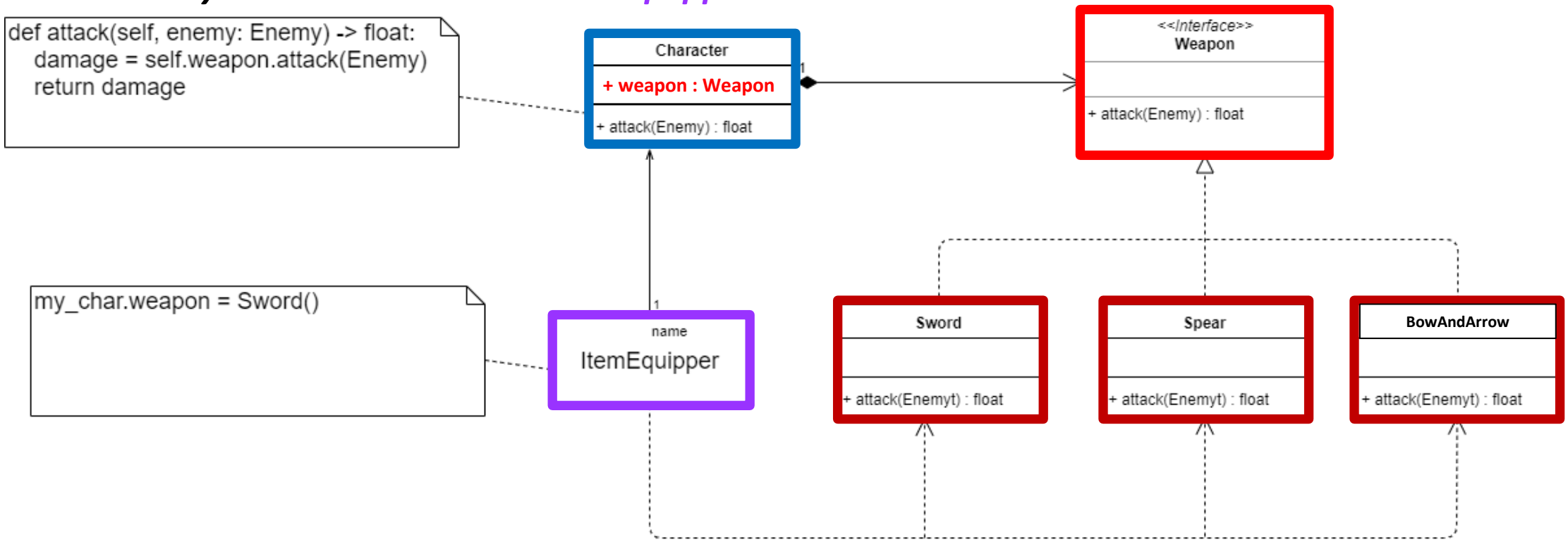
3) Have the **Character** class store an attribute that holds a reference to a **Base Weapon**. We can replace this weapon (**parent**) with any of the different ones (**child**) during run time.
Liskov Substitution Principle



Strategy Scenario – How do we change the weapon?

A separate client class can change the weapon of the character during runtime.

Let's say this class was called ItemEquipper



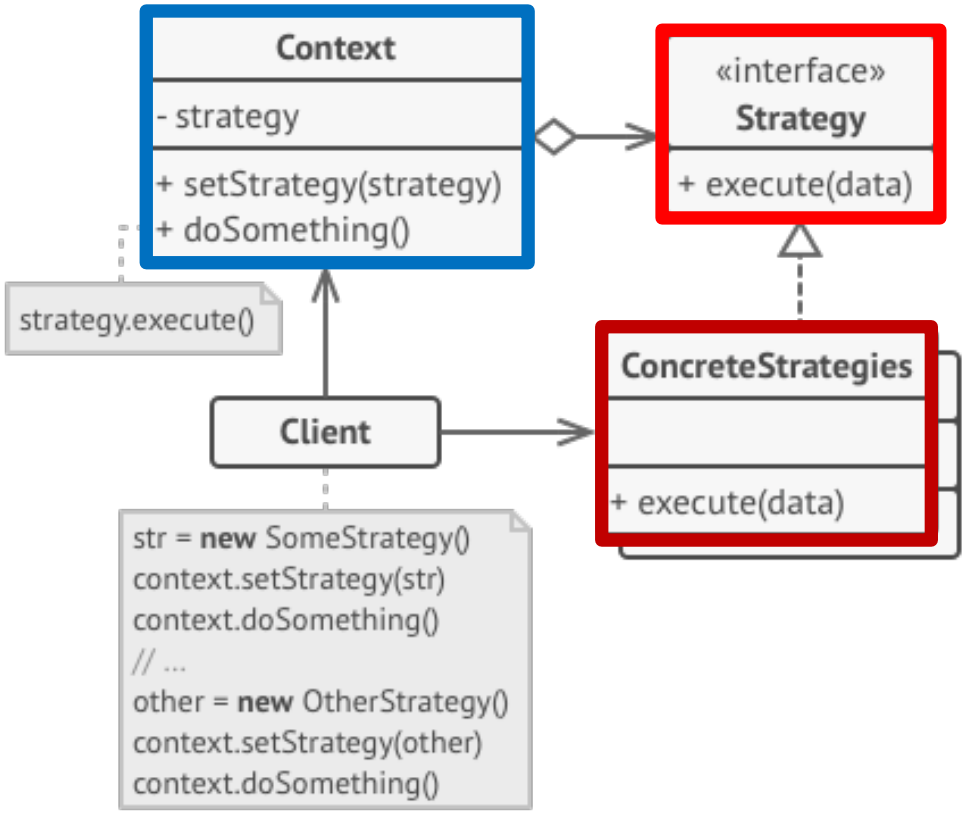
Strategy

Context is the original class that makes use of the **behaviour/strategy**.

In the previous example:

- the **context** was the character.
- the **behavior/strategy** was the weapon

This is a fairly simple pattern.



NOTE: The method(s) in the **Context** and the method(s) in the **Strategies** don't need to have the same interface.

strategy_sample_code.py

Strategy: When and Why do we use it

- When you want to use a **different algorithm for different use cases**. You can also swap them at run time
- Adheres to the open closed principle. We can **add new strategies without modifying the context**
- Replace inheritance with **composition** (similar to bridge pattern)
- Isolate the implementation of a strategy from the code that uses it.

Strategy– Disadvantages

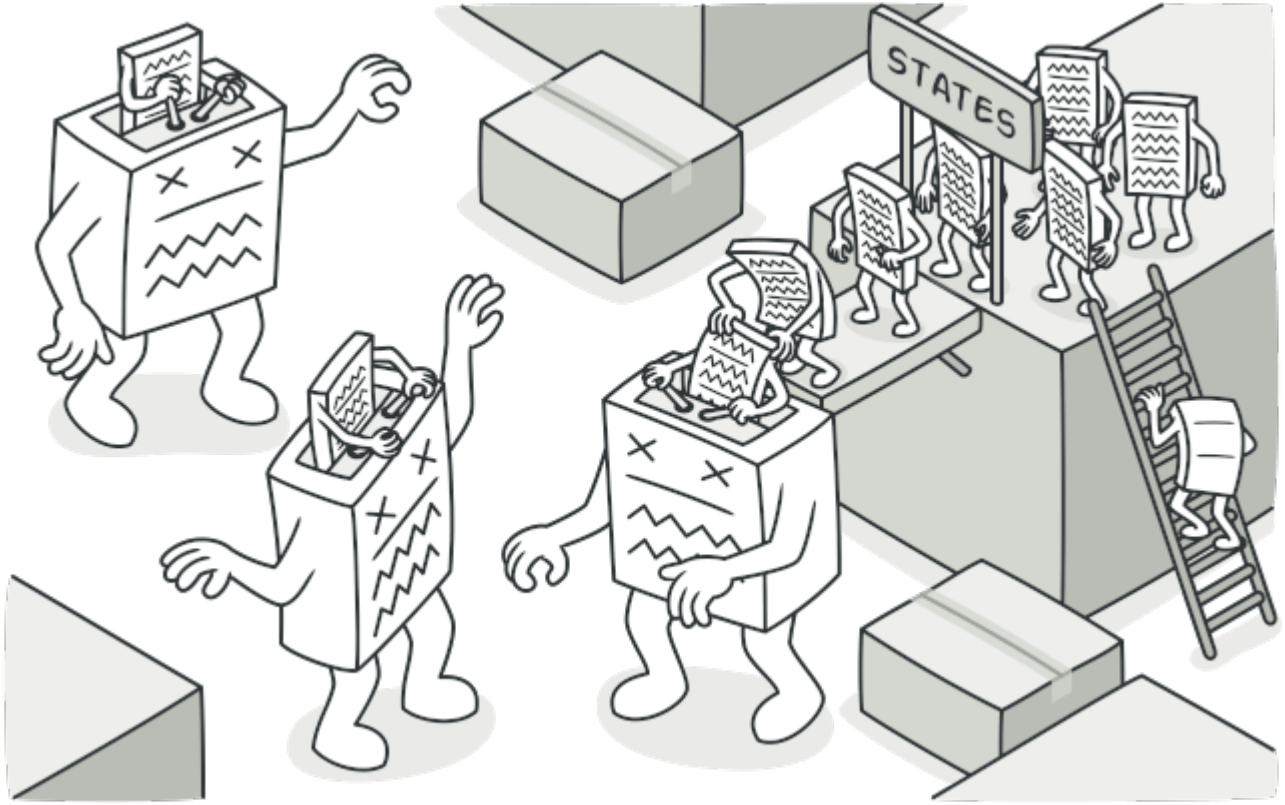
- It’s overkill if you only have a **few algorithms** that rarely switch.
- **Client** needs to be **aware of the different strategies** to be able to select between them.

State

WHEN A CLASS IS AMBITIOUS AND WANTS TO BE MANY THINGS

State

- A quick recap: A **state** of an **object** is defined as the **value and behaviours** of the object at any given time during program execution.
- A pattern implemented when an object wants to alter its behaviour. It might **behave like a different class**.
- Depending on its internal state, an object may react differently to different scenarios.
- The **state pattern** enables a **modular** way to **add states** and **control transitions**. We decouple the state from the object.



Apply this concept to objects

An object can be in multiple states. Its behaviour is usually determined by its state:

- **Happy State:** It wags its tail, sticks out its tongue and is playful
- **Scared State:** Tail between its legs, tongue inside its mouth, whimpering
- **Aggressive State**: Tail is not wagging, growling, tail is raised high, bared teeth

Dog

- **Hover State:** might change color
- **Pressed:** Might change image and how its rendered
- **Idle:** Render default image
- **Disabled:** Button is not rendered

A UI Button:

State – Naïve Implementation

A **ButtonEnum defines the different states a button can take.**

A button stores its **current state as an attribute**

We use a series of **if-else statements to change behaviour.**

While this may be acceptable for simple scenarios, it isn't maintainable in larger systems.

```
import enum


class ButtonEnum(enum.Enum):
    IDLE = 0,
    HOVER = 1,
    PRESSED = 2
    DISABLED = 3
```

```
class Button:
    def __init__(self, idle_img, hover_img, pressed_img, callback):
        self.btn_state = ButtonEnum.IDLE
        self.btn_on_click_callback = callback
        self.idle_img = idle_img
        self.hover_img = hover_img
        self.pressed_img = pressed_img

    def render(self):
        if self.btn_state == ButtonEnum.IDLE:
            print(f"Displaying {self.idle_img}")
        elif self.btn_state == ButtonEnum.HOVER:
            print(f"Displaying {self.hover_img}")
        elif self.btn_state == ButtonEnum.PRESSED:
            print(f"Displaying {self.pressed_img}")
        else:
            print("Button disabled")

def on_button_press_callback():
    pass

def main():
    idle_img = "btn_save_idle_sprite.png"
    hover_img = "btn_save_hover_sprite.png"
    pressed = "btn_save_pressed_sprite.png"
    button = Button(idle_img, hover_img, pressed, on_button_press_callback)
    button.render()
    button.btn_state = ButtonEnum.PRESSED
    button.render()
```

State – Naïve Implementation

We want to be able to add and remove states without re-compiling the button class.

We want to be able to implement state-dependent behaviours easily as well.

In other words…

Making any changes to the states will affect the Button class.

Button is coupled to its state.

We want the button to be decoupled from its state.

```python
import enum


class ButtonEnum(enum.Enum):
    IDLE = 0,
    HOVER = 1,
    PRESSED = 2
    DISABLEd = 3
```

```python
class Button:
    def __init__(self, idle_img, hover_img, pressed_img, callback):
        self.btn_state = ButtonEnum.IDLE
        self.btn_on_click_callback = callback
        self.idle_img = idle_img
        self.hover_img = hover_img
        self.pressed_img = pressed_img

    def render(self):
        if self.btn_state == ButtonEnum.IDLE:
            print(f"Displaying {self.idle_img}")
        elif self.btn_state == ButtonEnum.HOVER:
            print(f"Displaying {self.hover_img}")
        elif self.btn_state == ButtonEnum.PRESSED:
            print(f"Displaying {self.pressed_img}")
        else:
            print("Button disabled")

def on_button_press_callback():
    pass

def main():
    idle_img = "btn_save_idle_sprite.png"
    hover_img = "btn_save_hover_sprite.png"
    pressed = "btn_save_pressed_sprite.png"
    button = Button(idle_img, hover_img, pressed, on_button_press_callback)
    button.render()
    button.btn_state = ButtonEnum.PRESSED
    button.render()
```

State Pattern

We separate the **State** out into its own hierarchy.

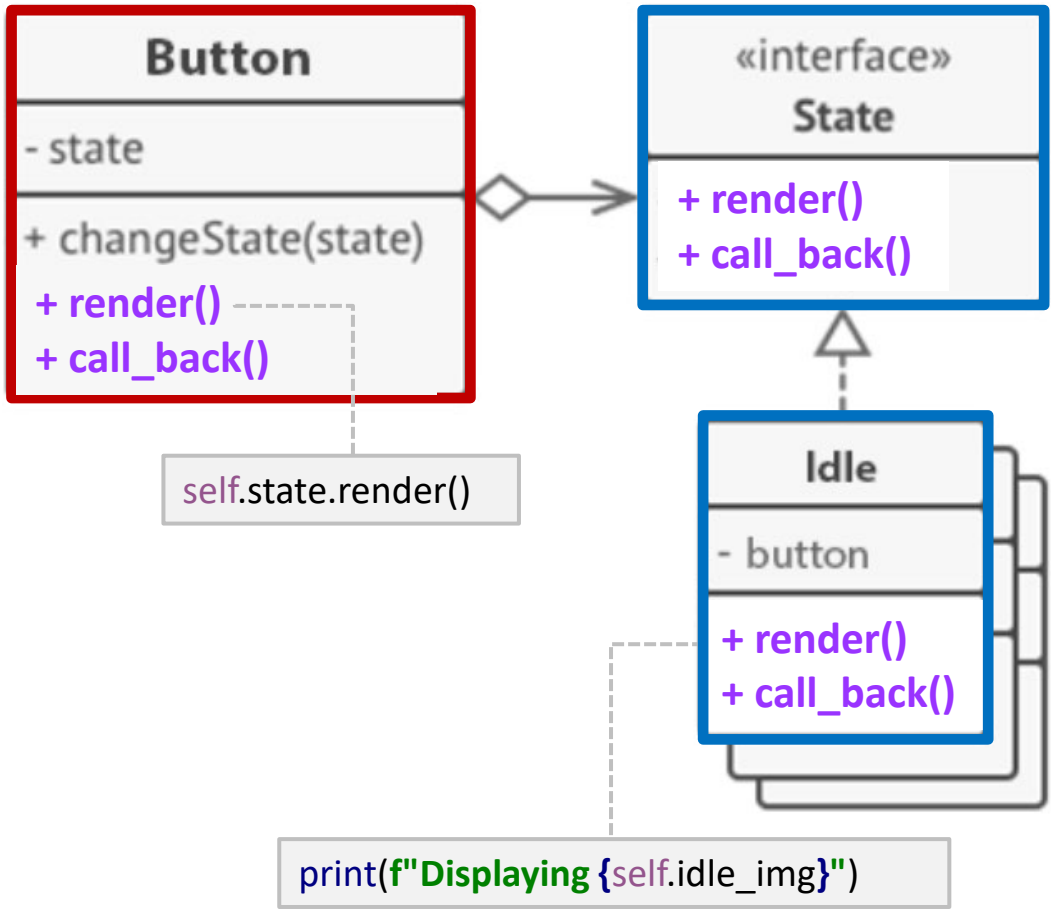
The **Button** should be dependent on a **State** interface, not the concrete states (Idle, Hover, Pressed, Disabled).

Now we can add and remove states easily.

The **State** Interface consists of all the **state-defined behaviours** in the **Button** class. This allows the Button class to communicate with it.

(In a way the Button wraps around its state)

Optionally, the state can keep a reference to the Button.



The generalized pattern

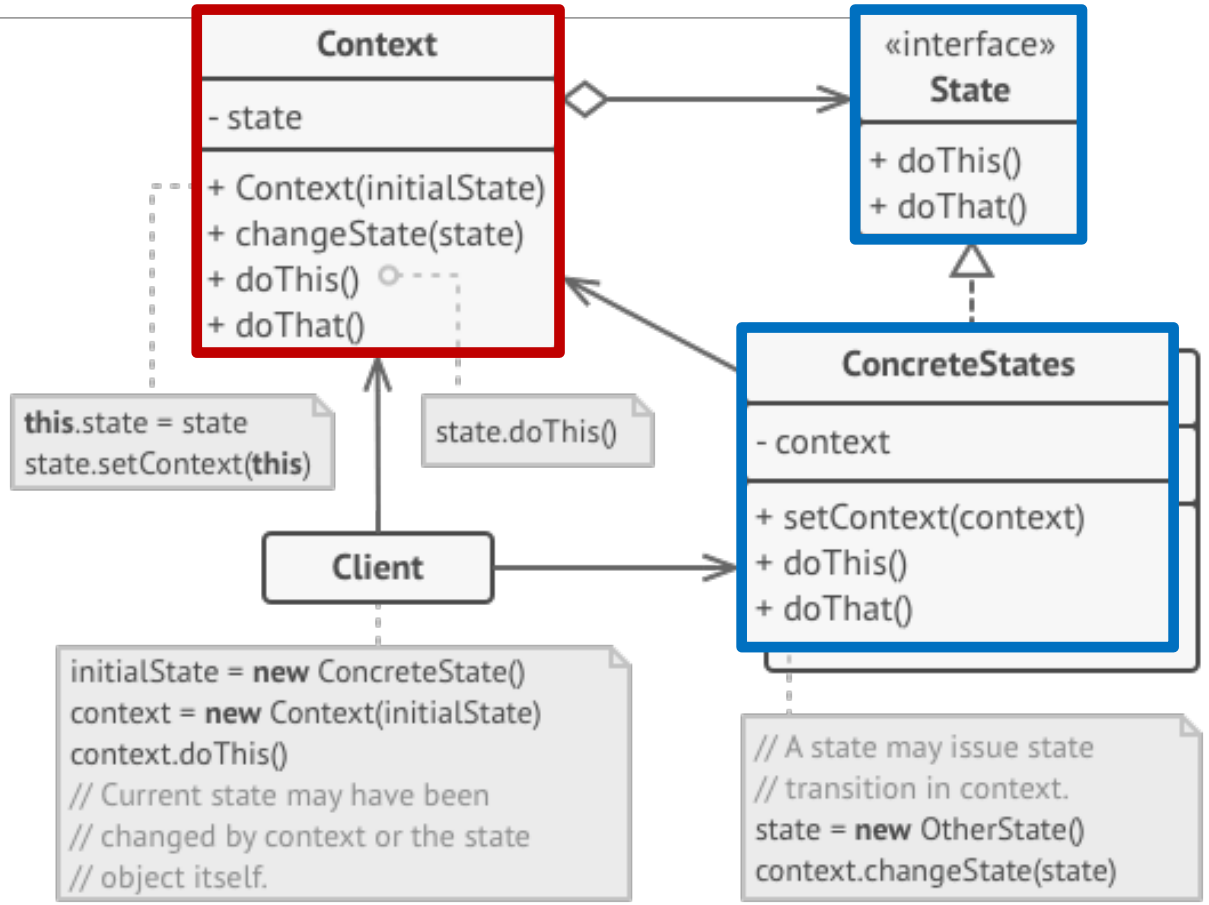
The object that has multiple **States** is called the **Context**

The **Context** has a method to change its state.

The **Context** delegates part or all of its behaviour to the **State**.

The concrete **State** may or may not have access to the **Context**. If the **State** is responsible for changing the **State** of the **Context** then this may be required.

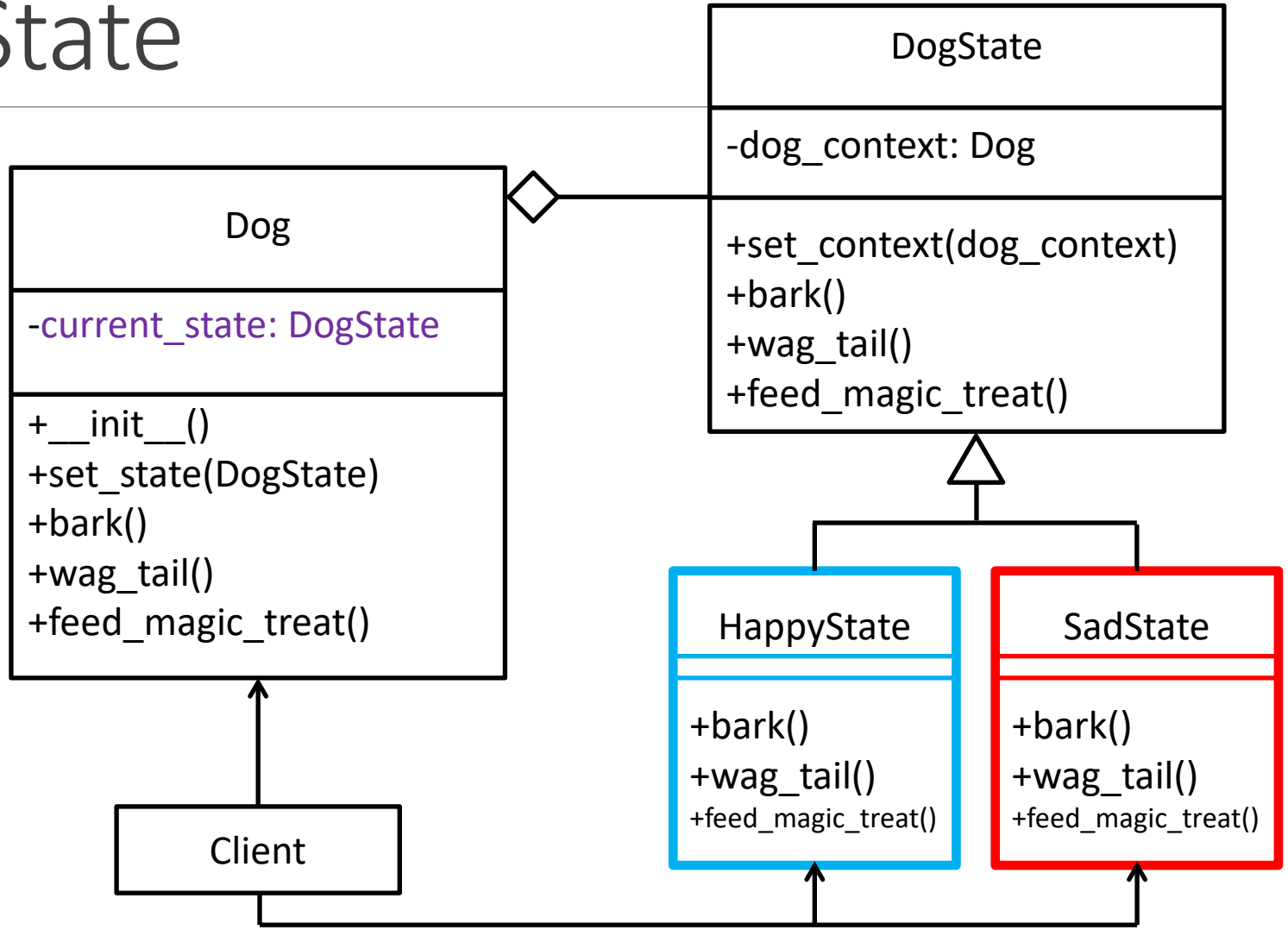
The client instantiates the new **State** and assigns it to the **Context**.



Behavioural – State

Create a system to simulate a dog behaving differently depending on its mood.

Happy – happily barking, wagging tail
Sad – whimpering, tail between legs



dog_state.py

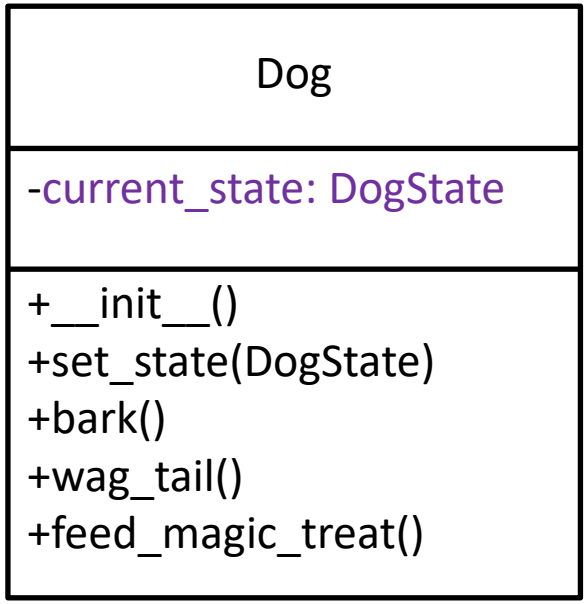
Behavioural – State

We'll demonstrate the State pattern using our Dog example. The goal over the next few slides is:

1. Create a **Dog**
2. Set the **Dog** to the HappyState
3. Make it perform some actions in the HappyState
4. Set the **Dog** to the SadState via a HappyState method
5. Make it perform some actions in the SadState

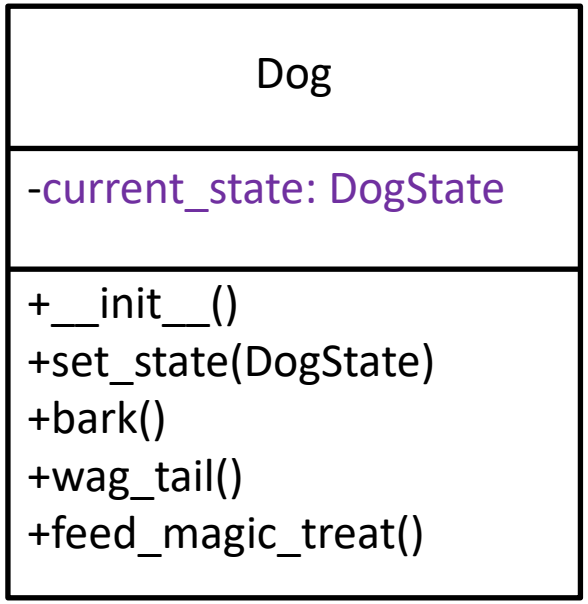
Behavioural – State

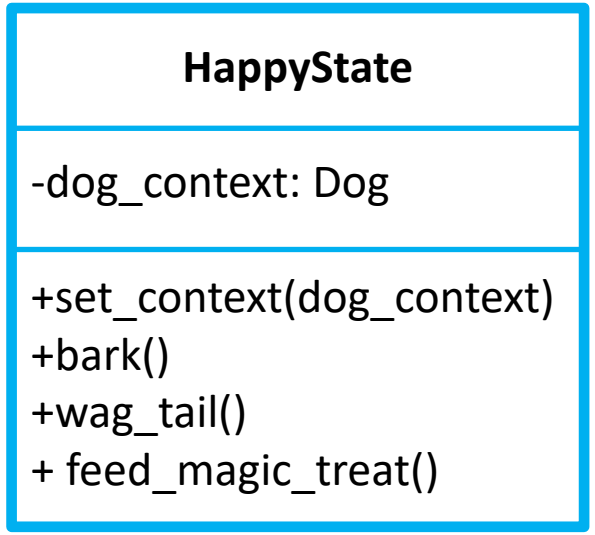
Client instantiates **Dog** object



Behavioural – State

Client instantiates **HappyState** object



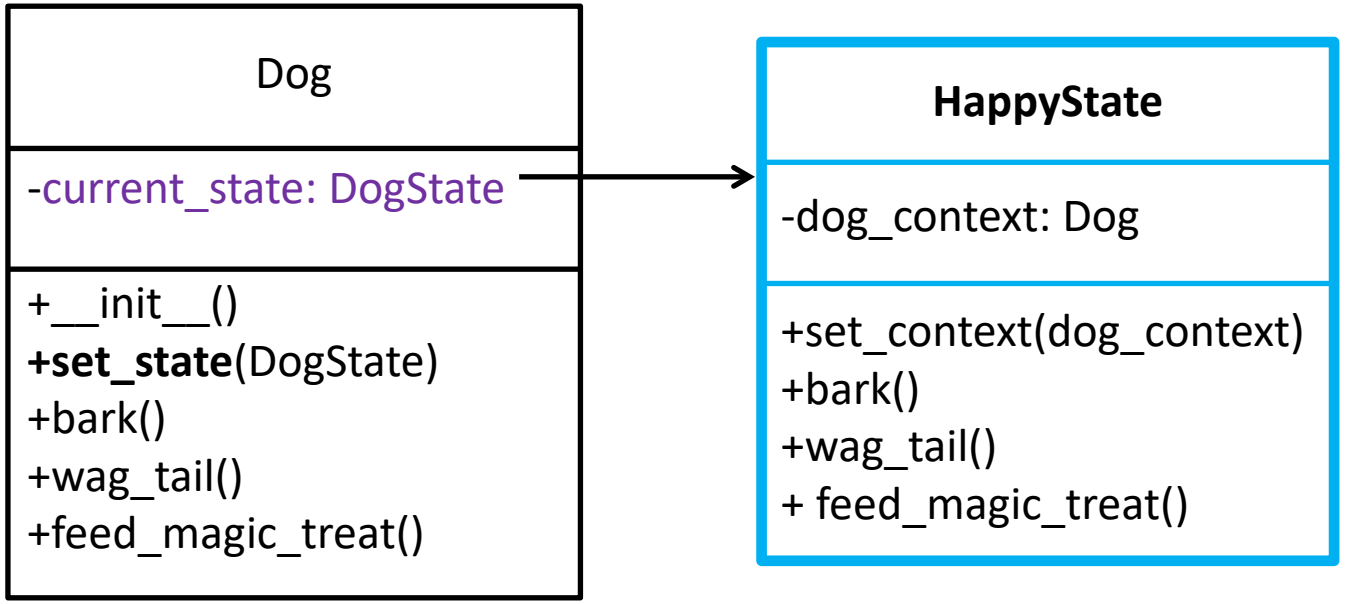


Behavioural – State

Client sets **Dog** object's state to **HappyState** object

Code in **set_state** method**:**

- makes current_state point to HappyState object

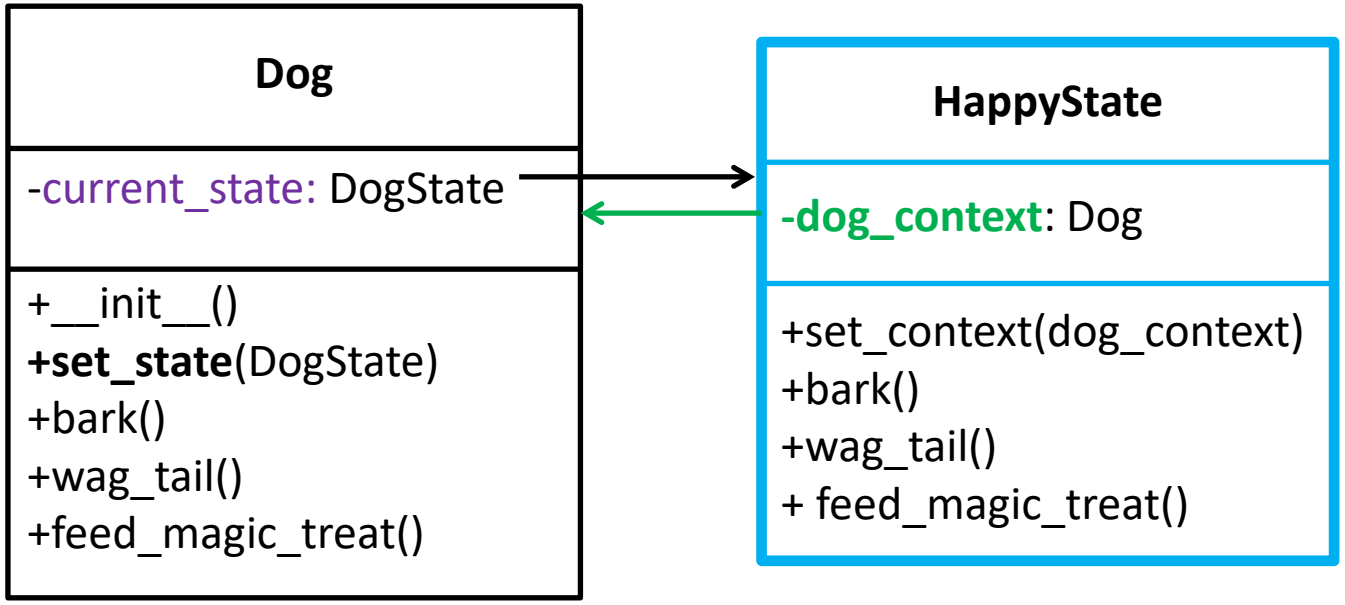


Behavioural – State

Client sets **Dog** object's state to **HappyState** object

Code in **set_state** method**:**

- makes current_state point to HappyState object
- also makes HappyState object's **dog_context** point to **Dog** object
- This **two way communication** is needed later when HappyState calls Dog to change future states.

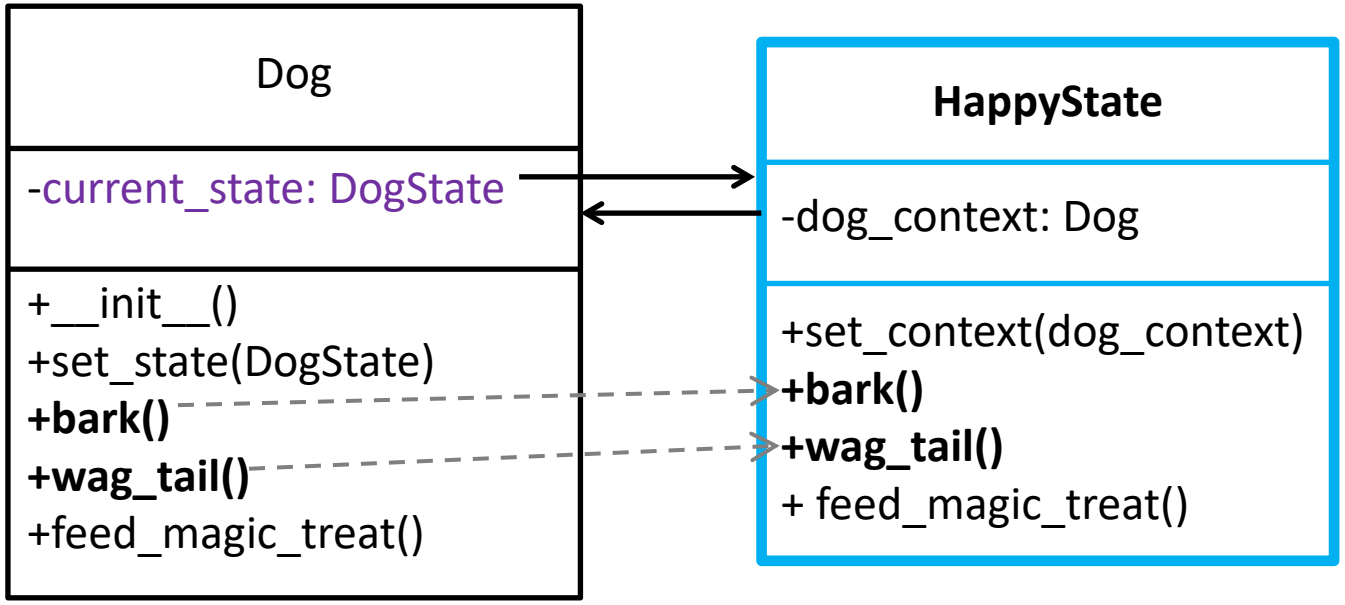


Behavioural – State

Dog is now set to HappyState :) Let's make it perform some actions

Client calls Dog's **bark()**, **wag_tail()** methods

- Code in these methods uses current_state to call **HappyState**'s **bark()**, **wag_tail()** methods
- Output is "Happy bark" and "Happy tail wag"



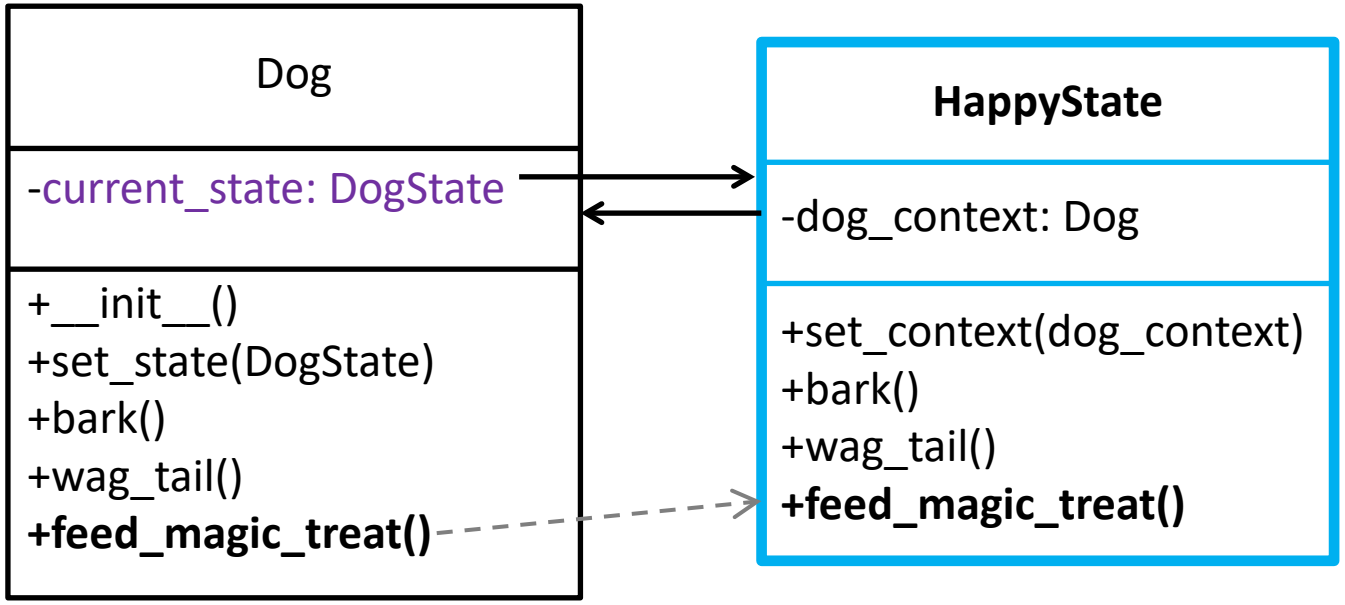
Behavioural – State

Now we want to demonstrate that States can call Contexts to change to other States.

Let's change Dog's **HappyState** to **SadState** using the **feed_magic_treat()** method

Client calls Dog's **feed_magic_treat()**

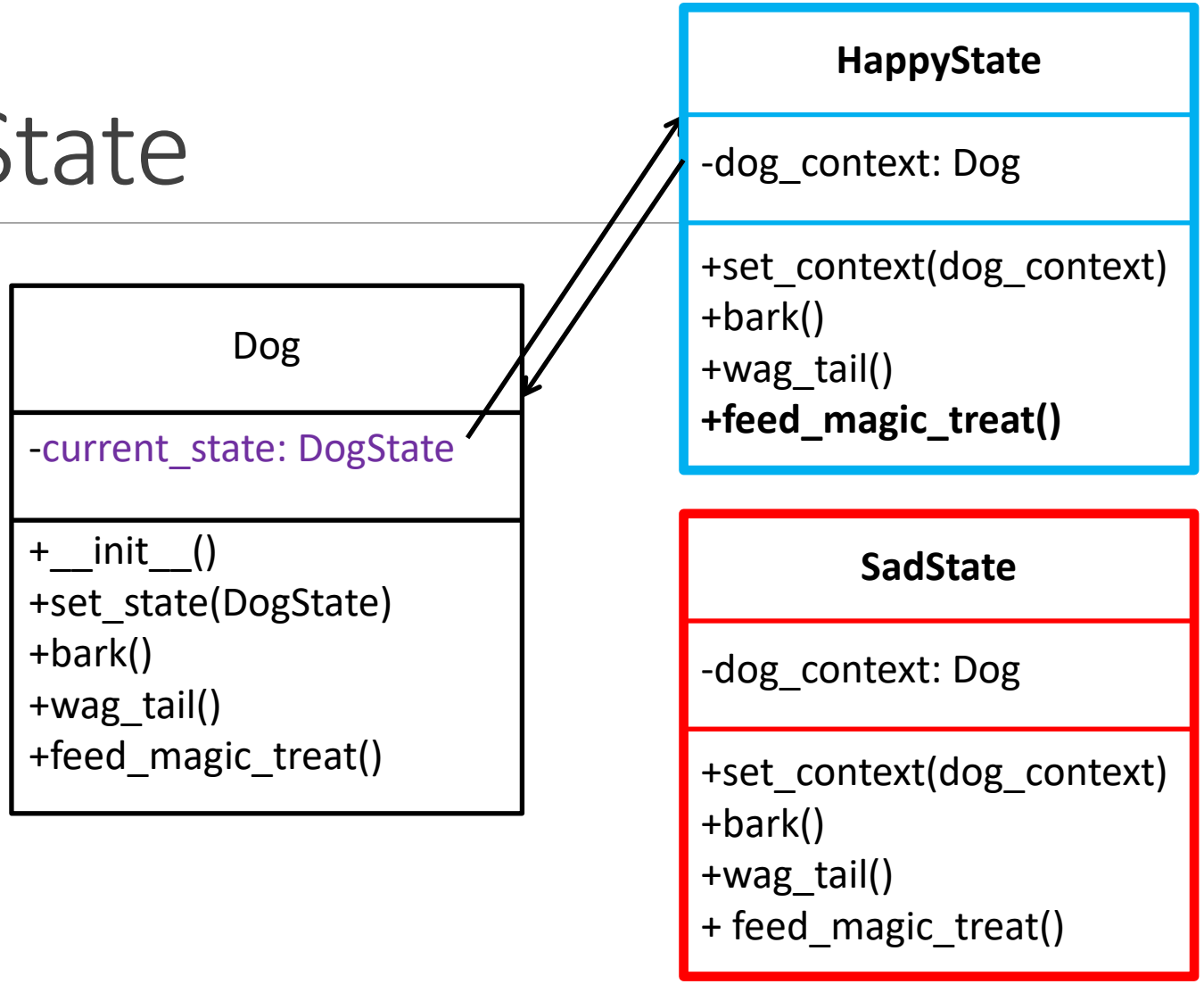
- This calls HappyState's **feed_magic_treat()** method



Behavioural – State

Code in HappyState's **feed_magic_treat()** method:

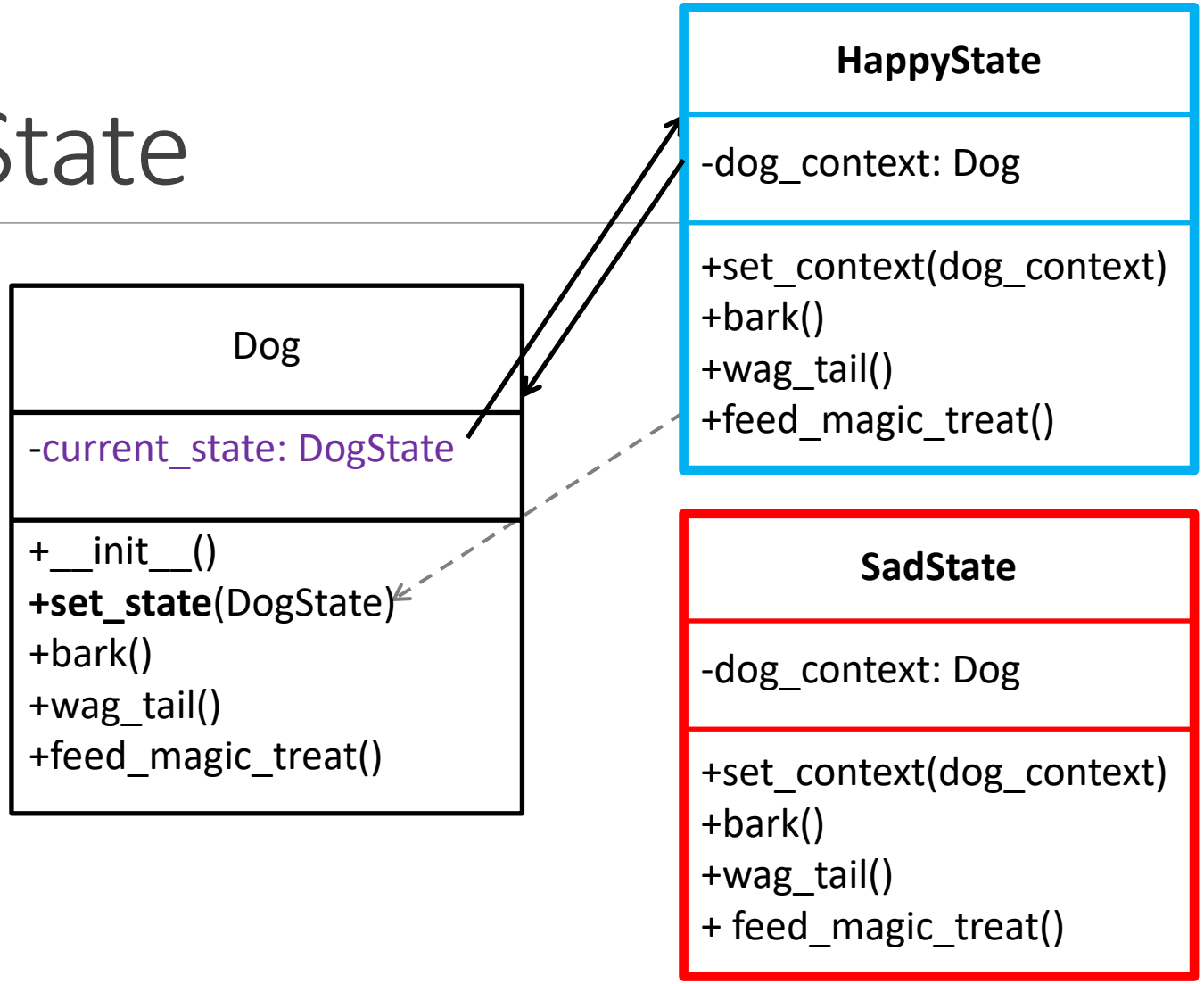
- Instantiates **SadState**



Behavioural – State

Code in HappyState's **feed_magic_treat()** method:

- Instantiates **SadState**
- Calls Dog's **set_state** method

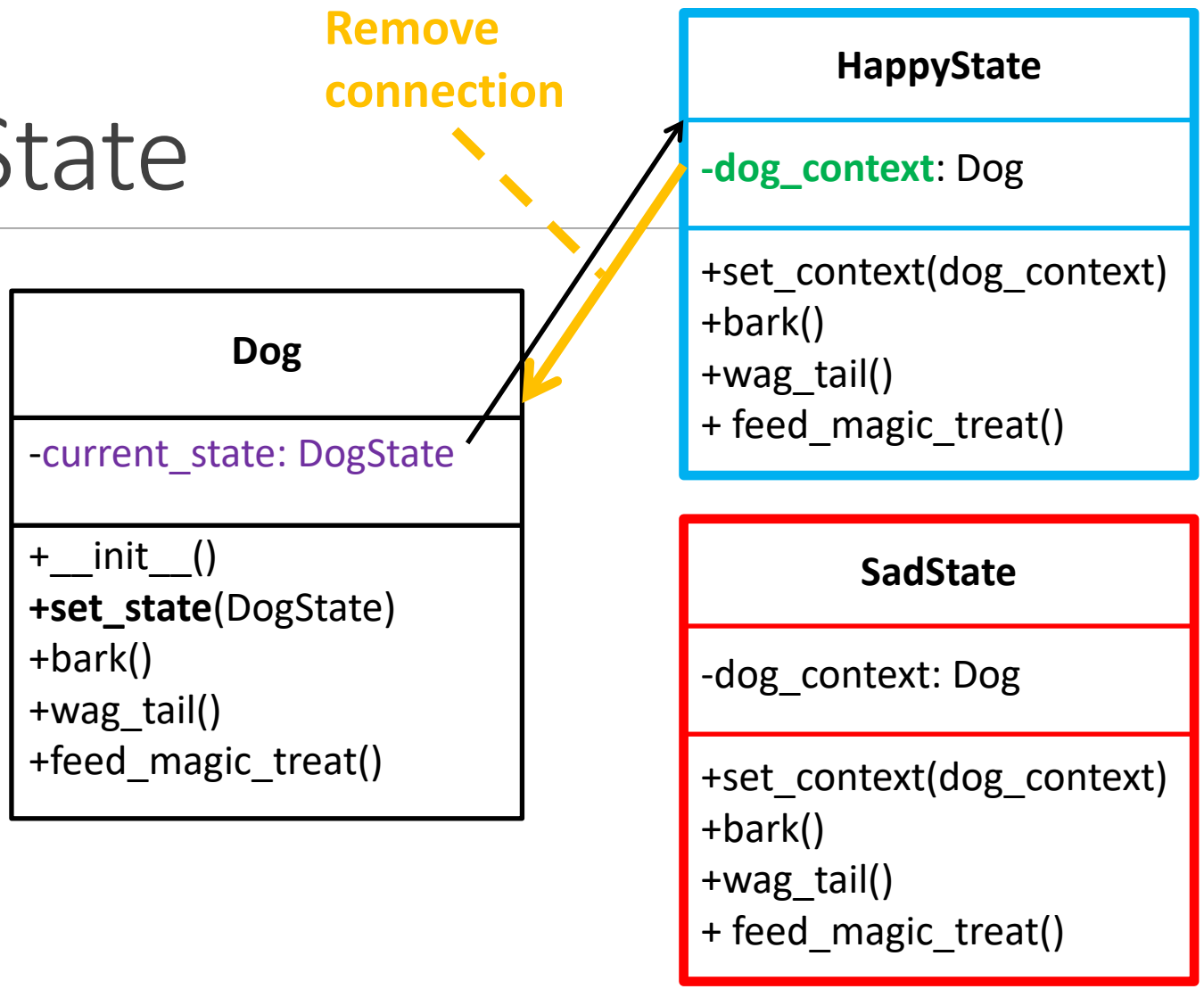


Behavioural – State

Before changing to a new **SadState**, the **Dog** needs to remove connections to the previous **HappyState**

Code in Dog's **set_state** method:

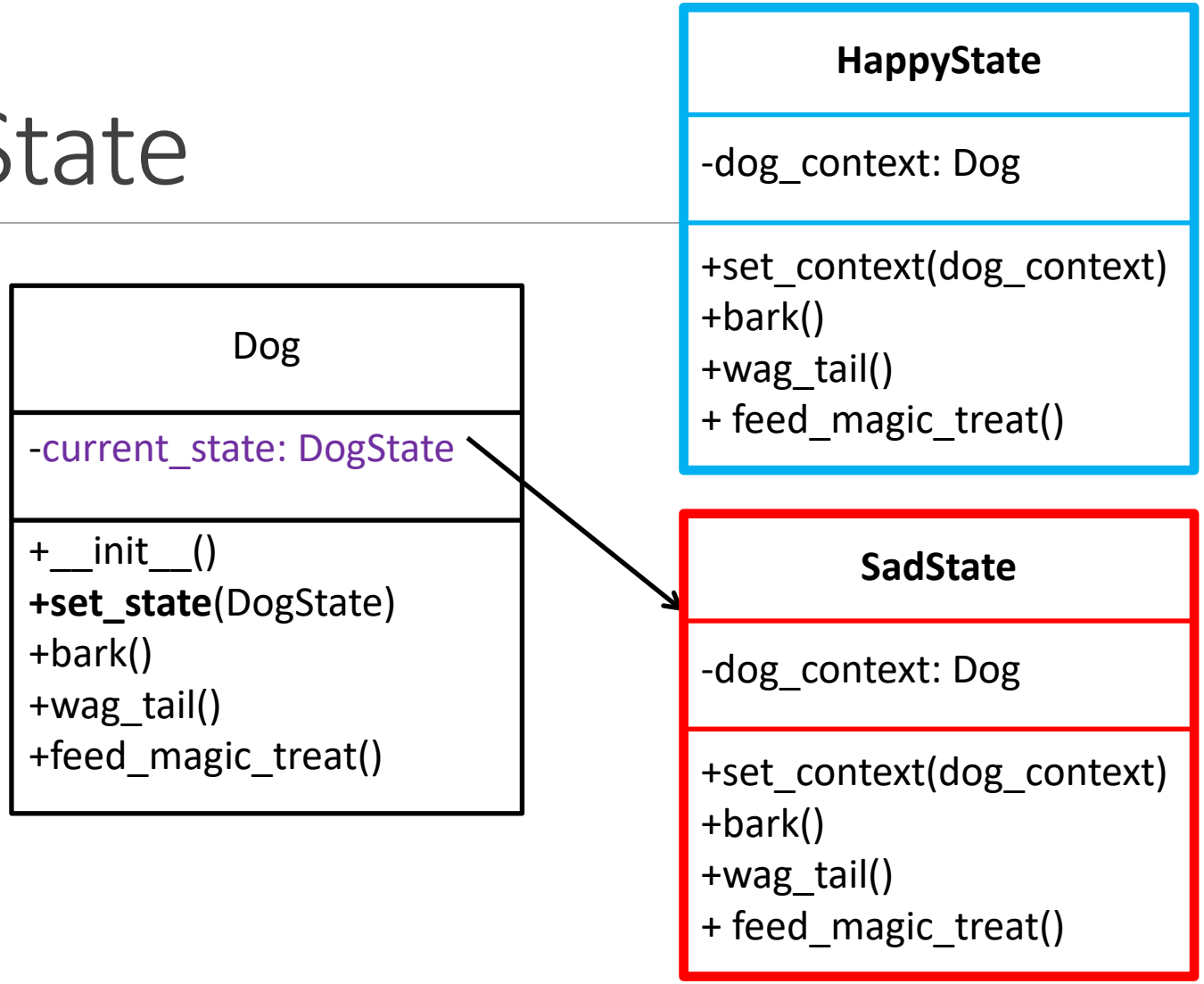
- **Removes** **HappyState**'s **dog_context** connection to **Dog** by setting it to None



Behavioural – State

Code in Dog's **set_state** method:

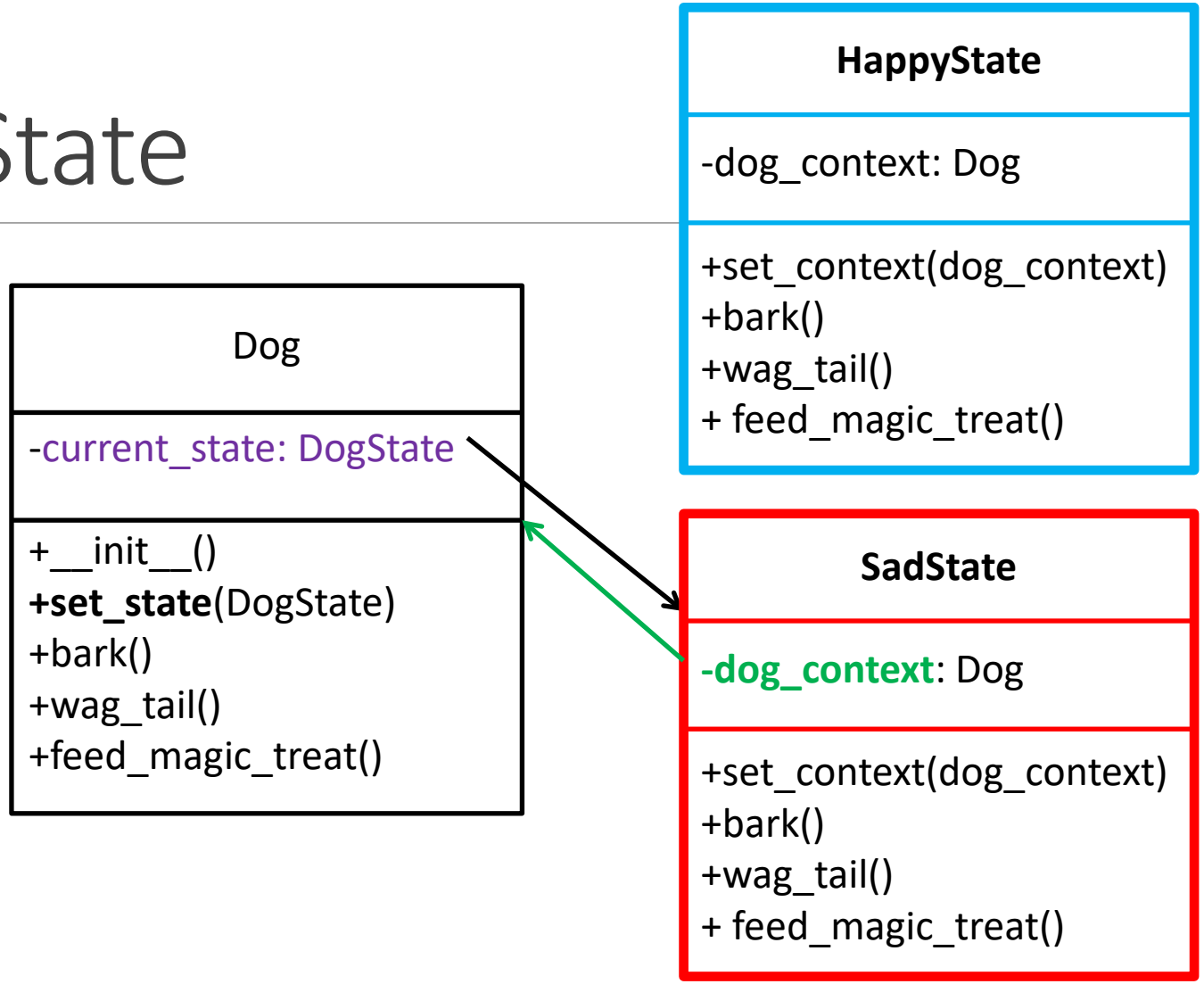
- Sets current_state to point to **SadState**



Behavioural – State

Code in Dog's **set_state** method:

- Sets current_state to point to **SadState**
- Sets **SadState's dog_context** to point back to Dog
- This **two way communication** is needed later when SadState's calls Dog to change future states.

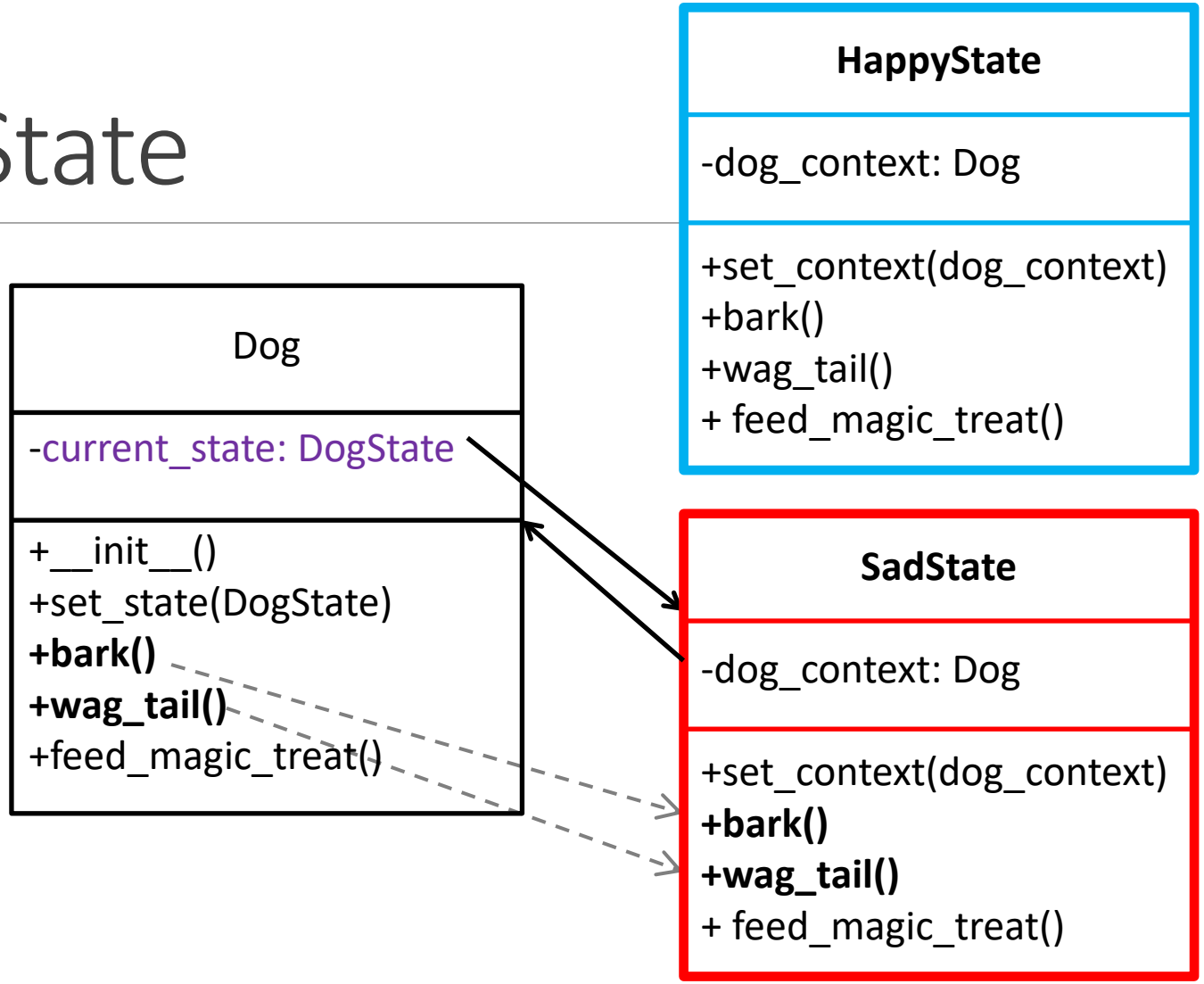


Behavioural – State

Dog is now set to **HappyState :(** Let's make it perform some actions

Client calls Dog's **bark()**, **wag_tail()** methods

- Code in these methods use current_state to call **SadState**'s **bark()**, **wag_tail()** methods
- Output is "Sad bark" and "Sad tail wag"



dogState.py

State: Why and When do we use it

- Use this pattern when an object's behaviour is **heavily dependent on its states** and the number of states can be large (or you may want to add more in the future)
- Avoid massive if statement blocks to handle object behaviours based on state conditions.
- You can create **hierarchies of state** classes if there are a lot of similar **states that share code**.
- Implements the **Single Responsibility Principle**. Each state is self-contained.
- Implements the **Open/Closed Principle**. We can add and remove states without modifying the context and multiple if-else statements.

State – Disadvantages

- Can become **hard to maintain if the object rarely changes state** or only has a few states. Sometimes a simpler solution involving conditional statements works better.
- State Transitions can be complex, especially if **each state knows about its neighbouring states** (that it transitions from/into). This can make the **states highly coupled** with each other.
- There are a lot more classes and code to maintain.

How is State different from Strategy?

They solve different problems

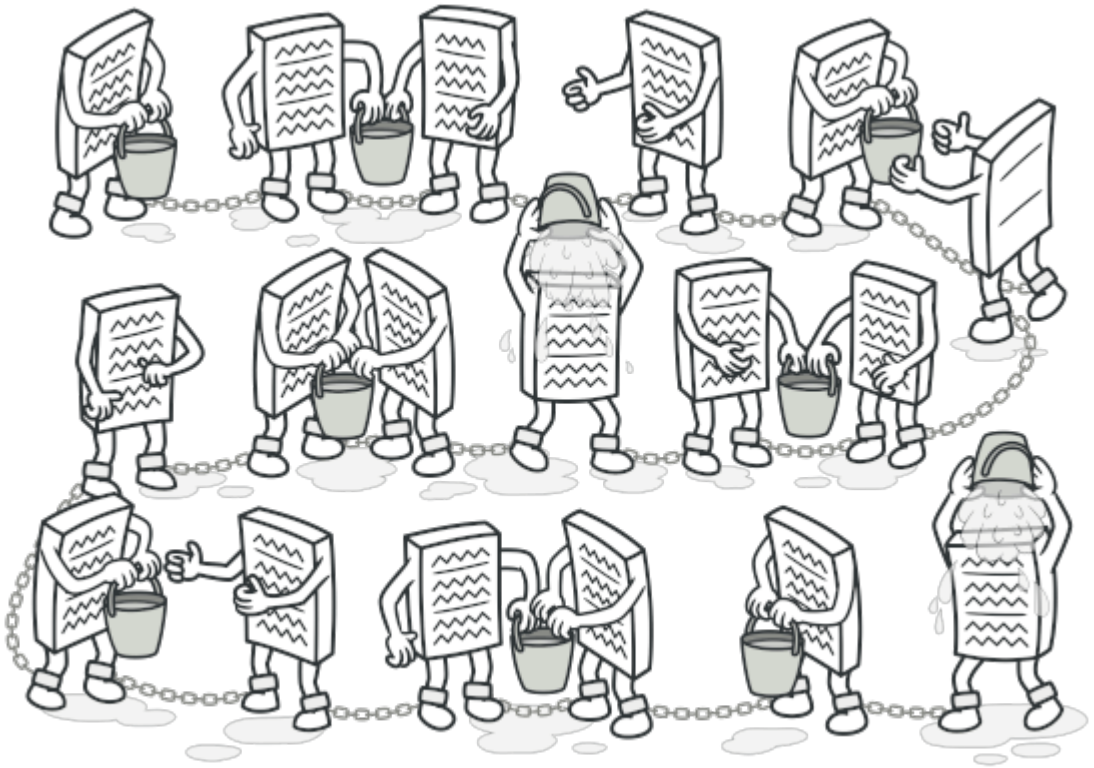
Strategy - select different algorithms or behaviors at runtime

- Strategies **don't know** about each other
- Context and Strategy **interfaces don't have to match**
- Strategy usually does not have a reference to context

State - change behavior when internal state changes. Often based on some rules prescribed by a finite state machine.

- States **can know** about each other
- Context and State **interfaces mostly match**
- State can have a reference to context

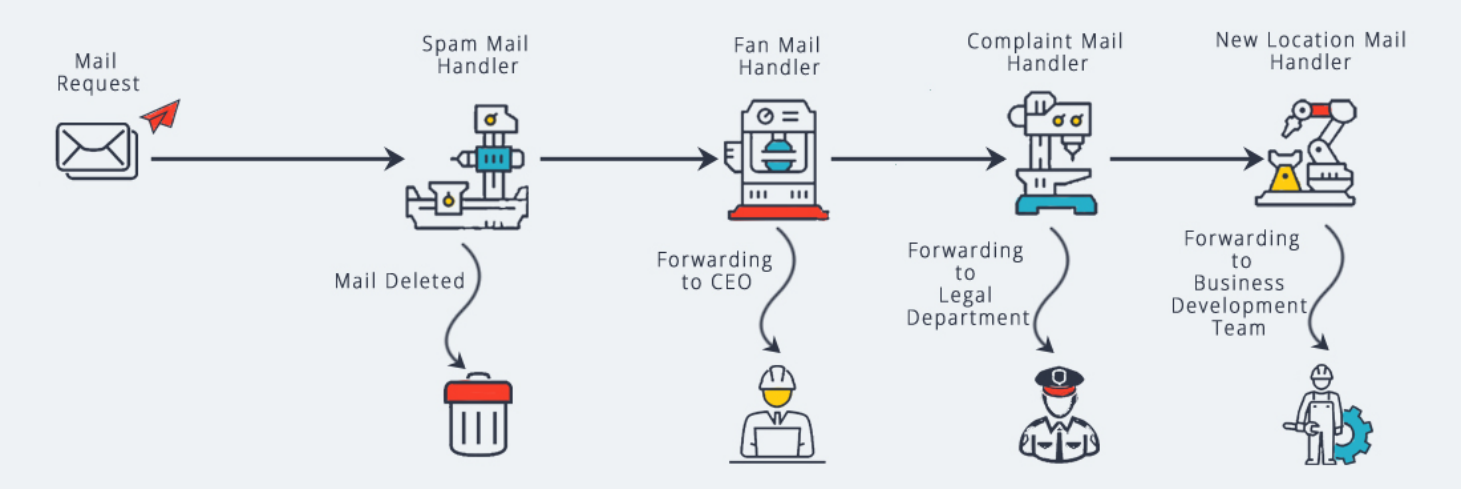
Chain of Responsibility



WHEN YOU WANT TO HAVE A LOT OF RESPONSIBILITIES BUT STILL BE FLEXIBLE

Chain of Responsibility

- When an object or a set of objects needs to undergo different **"steps" of processing**.
- These could be **checks**, **validation**, **formatting**, **security**, **setup**, **etc**.
- Set up a series of **Handlers**. Each handler is unique but **implements the same interface** and does something to a **Request**.
- Each Handler **has a reference to another handler** and it may, depending on its code, pass on the request to another handler.



Chain of Responsibility - Scenario

Say there was an old fashioned school that used paper forms.

Students had to fill out an **Enrolment Form** every semester listing the courses they wanted to enrol in.

This form undergoes a series of validation checks, and get's processed by hand.

Student Enrolment Form

Hogwarts School of Witchcraft and Wizardry

This form needs to be filled by anyone students in years 3 – 7. Students in years 1 and 2 have their subjects predetermined.

Name: _____

Student ID: _____

Age: _____

Course Code	Course Name	Semester # (1-2)

Student Enrolment Form

- Eventually (Finally!) the school went digital with their administrative tasks.
- **Forms** were processed by their administrative system.
- The code to process their **enrolment form** kind of looked like this.
- This is terrible:
  - Multiple if statements in a **static order**

```
def process_enrolment(student_application_form: EnrolmentApplicationForm) -> (str, bool):
    validated = True
    student_record = database.get(student_application_form.student_id)

    if student_record.name != student_application_form.name:
        validated = False
        return "invalid record", validated

    if not student_application_form.fees_paid:
        validated = False
        return "fees not paid", validated

    for course in student_application_form.courses_for_enrolment:
        if not check_course_offering(course):
            validated = False
            return "course not offered", validated
        if student_record.age < course.min_age:
            validated = False
            return "age requirement not met", validated
```

…

Student Enrolment Form

- The system also processes other similar forms.
- Many of the forms share different **validation checks**.
- Sometimes the order in which they are done is changed depending on the form.
- Right now the code is duplicated and difficult to maintain. We have several big classes with redundant code.

```
def process_enrolment(student_application_form: EnrolmentApplicationForm) -> (str, bool):
    validated = True
    student_record = database.get(student_application_form.student_id)

    if student_record.name != student_application_form.name:
        validated = False
        return "invalid record", validated

    if not student_application_form.fees_paid:
        validated = False
        return "fees not paid", validated

    for course in student_application_form.courses_for_enrolment:
        if not check_course_offering(course):
            validated = False
            return "course not offered", validated
        if student_record.age < course.min_age:
            validated = False
            return "age requirement not met", validated
```

…

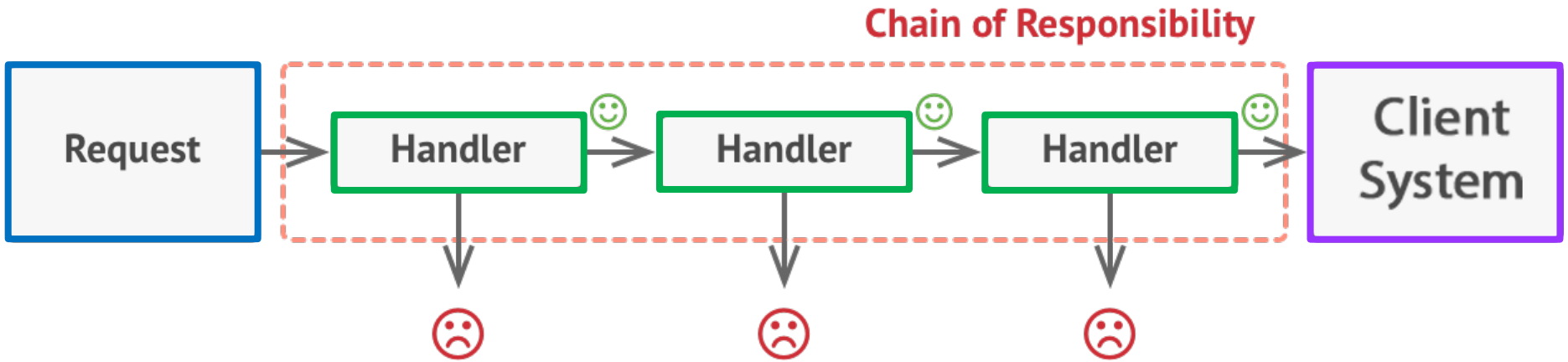
Chain of Responsibility

The Chain of Responsibility pattern **separates** these different **processing steps into different classes**. Each class is a **Handler**

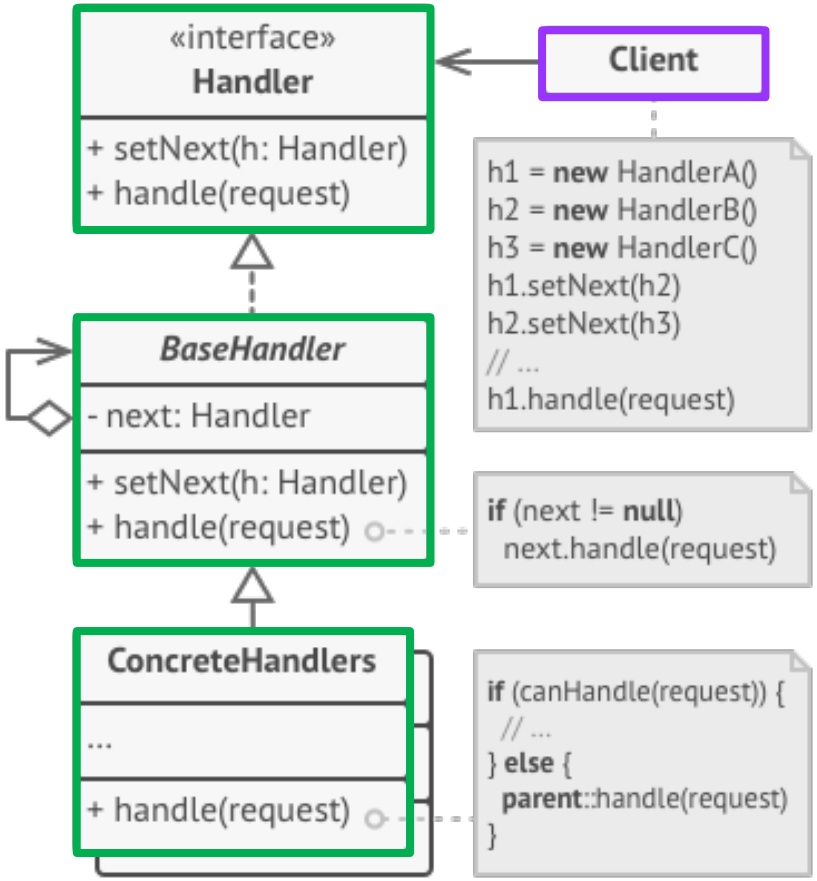
All these classes share the same interface, that is how one **handler** can pass on the **request** to another handler.

It is kind of like a **linked list of responsibilities** where each **handler** forms a node. We can arrange a different list for different scenarios.

The execution may stop midway and exit the chain if the**Handler** deems it necessary



Chain of Responsibility



Requirements:

- Each **Handler** implements the **same interface.**
- **BaseHandler** is an **Optional** parent class that can hold some duplicate code (such as **setNext(h: Handler)**)
- Each **handler** implements a **handle(request) method** which is where they carry out their specific code.

Let’s implement our enrolment form!

Or at least a part of it.

Let’s draw out the UML diagrams:

1 - **Enrolment Application Forms**
- **Student Account**

2 - 3 **handlers** to verify application forms:
- **StudentValidationHandler**
- **CheckCourseOfferingHandler**
- **FeesHandler**

3 - **Enrolment System**

Cheesy Teamwork Image for motivation

Let’s implement our enrolment form!

Or at least a part of it.

Let’s draw out the UML diagrams:

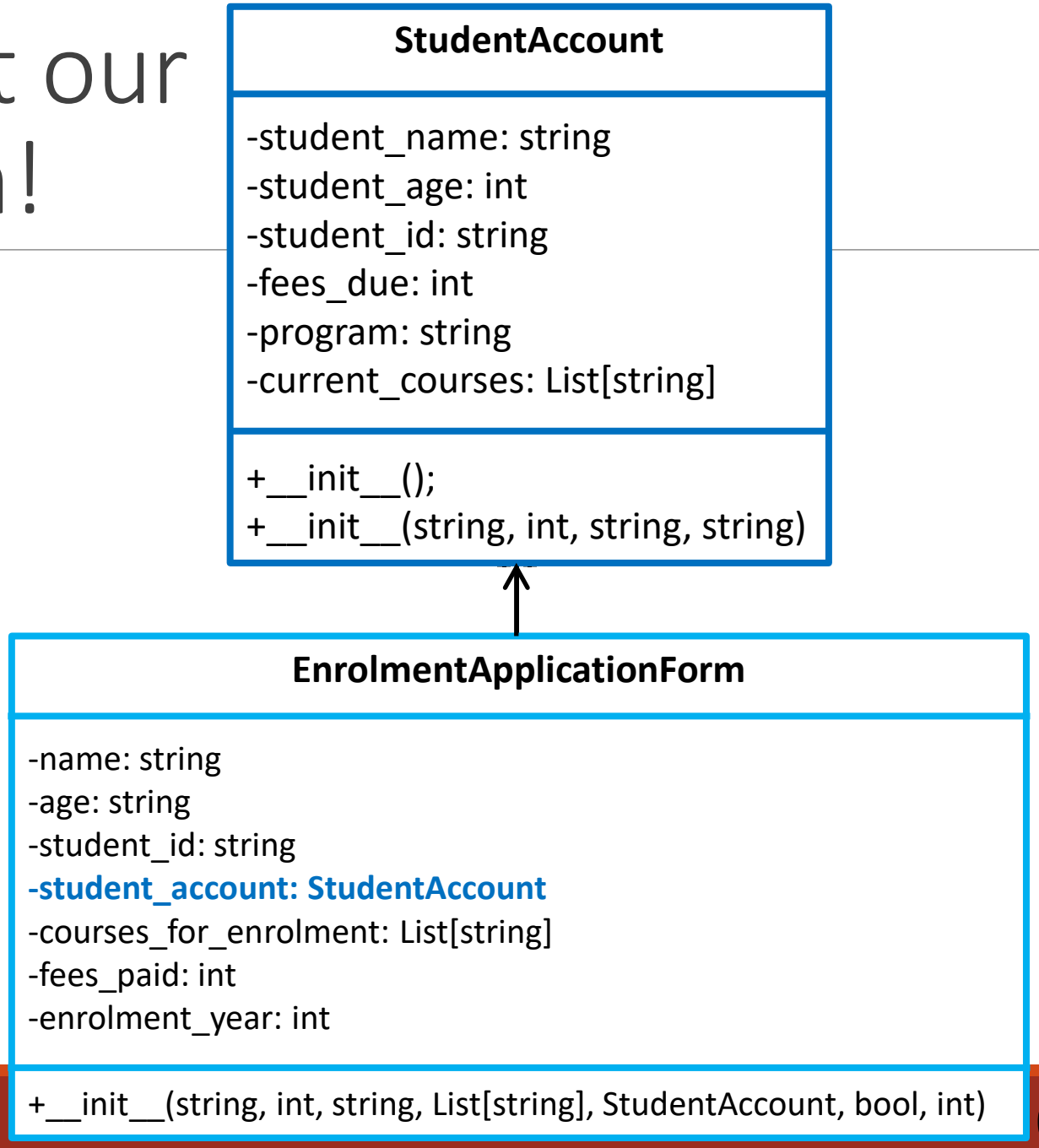
1 - Enrolment Application Forms

- **Student Account**

2 - 3 handlers to verify application forms:

- StudentValidationHandler
- CheckCourseOfferingHandler
- FeesHandler

3 - Enrolment System



Let’s implement our enrolment form!

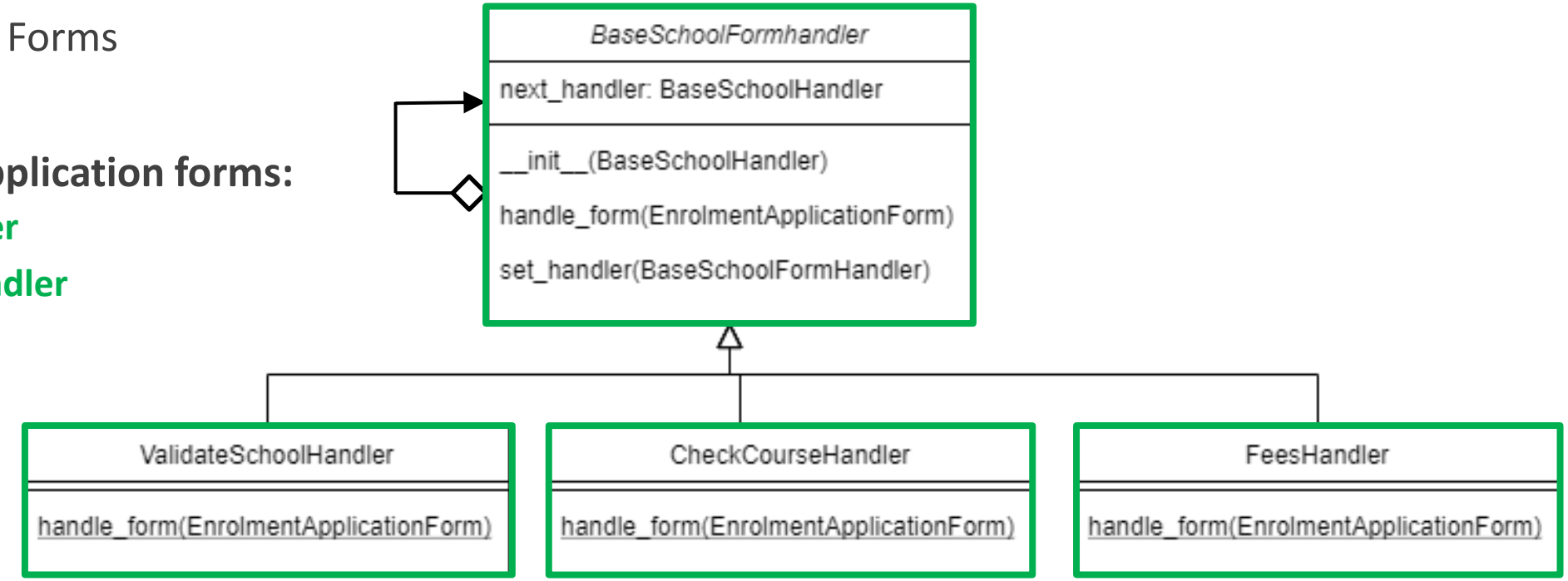
Or at least a part of it.

Let’s draw out the UML diagrams:

1 - Enrolment Application Forms
- Student Account

2 - 3 handlers to verify application forms:
- **StudentValidationHandler**
- **CheckCourseOfferingHandler**
- **FeesHandler**

3 - Enrolment System



Let’s implement our enrolment form!

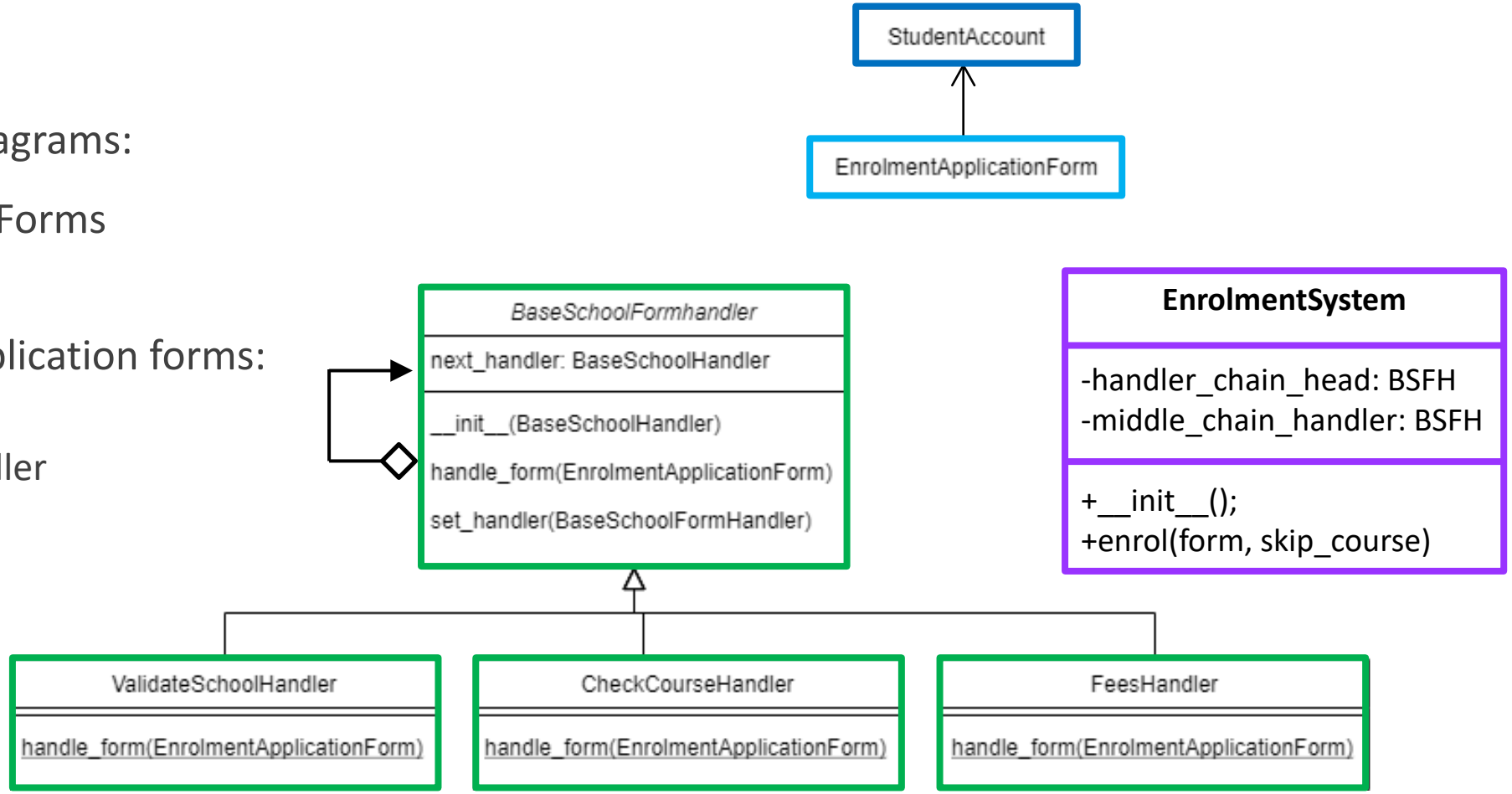
Or at least a part of it.

Let’s draw out the UML diagrams:

1 - Enrolment Application Forms
- Student Account

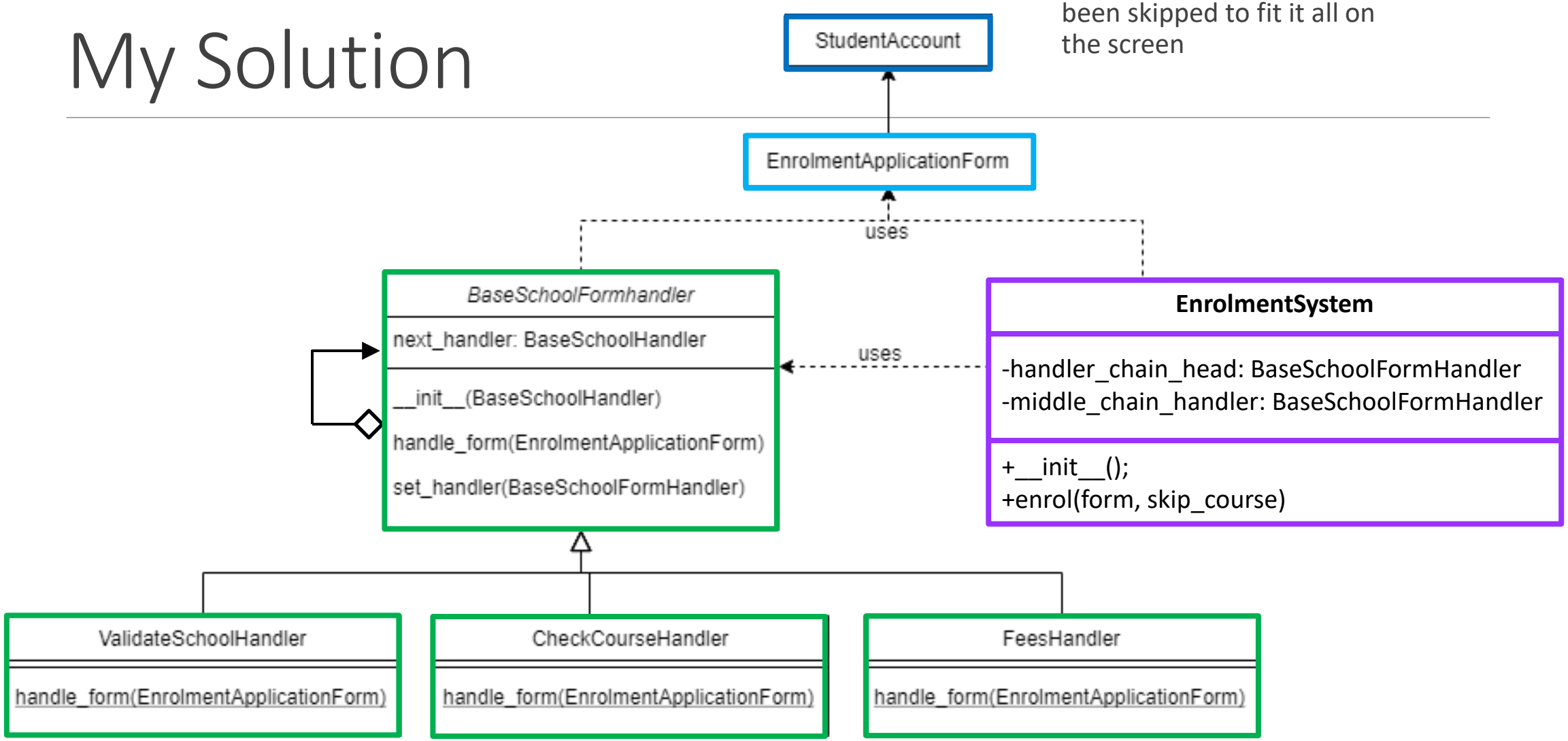
2 - 3 handlers to verify application forms:
- StudentValidationHandler
- CheckCourseOfferingHandler
- FeesHandler

3 - Enrolment System

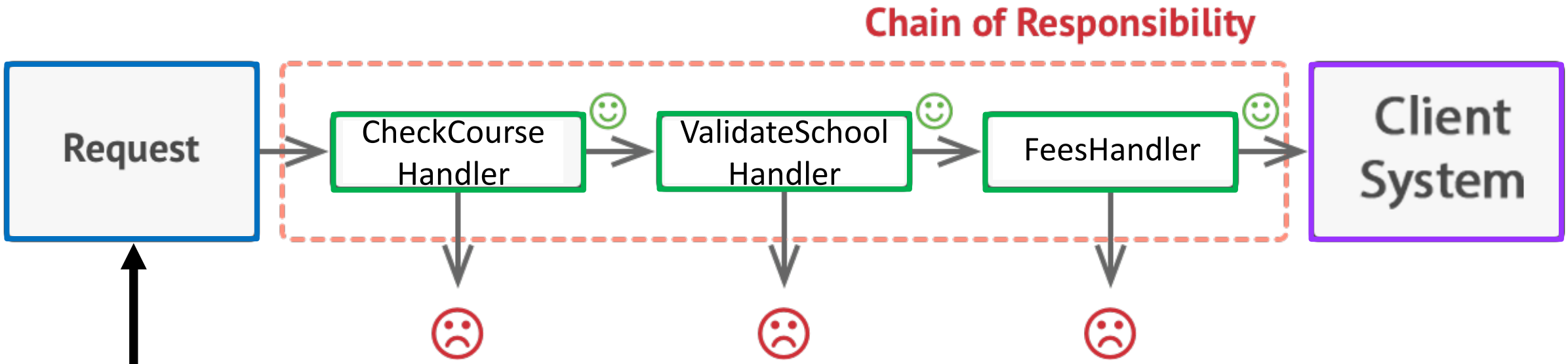


My Solution

NOTE: These would have attributes in them, but have been skipped to fit it all on the screen



My Solution



Client System
1. Create student account and forms
2. Send form to enrolment system

chain_of_responsibility.py

Chain of Responsibility: Why and When do we use it

- If your program is expected to **process different kinds of requests** in various ways.
- When you need to **process something in a particular order**.
- If the **sequence and ordering of request-processing** is not known before hand and needs to be **determined at run-time**. We can control the order of request handling.
- Single Responsibility Principle. Each handler does one thing. We have decoupled **classes that invoke operations** (E.g. **EnrolmentSystem**) from **classes that perform operations** (the **handlers**)
- Open/Closed Principle. We can introduce **new handlers without modifying existing handlers or client code**.

Chain of Responsibility – Disadvantages

- More classes to maintain.
- Some requests may end up unhandled. This may happen if we don't **set up the ordering of handlers properly**.